

现代网页开发入门指南书 - 从数据库设计，到后端，再到前端 v1.5版本-增量版

前言

如今的Web，早已不再是前后端混合的 PHP 世界。我们早已进入了前后端分离的，Web3.0的 - 未来。

欢迎你阅读本文 - 「现代网页开发入门指南书 - 从数据库设计，到后端，再到前端」

由 Dontalk 会员 - 【葵】 耗时4天编写 - 如有不对，敬请原谅和指正。

如果你希望下载本文 PDF 以及配套代码，请前往「dl.dontalk.org」找到「现代网页开发入门指南书 - 从数据库设计，到后端，再到前端」即可下载。或者博文提到的 GitHub地址 也可以下载到副本 (包括本指南书的PDF档)。

Dontalk 地址: dontalk.org

Github 仓库地址(25年5月中): <https://github.com/infoabcd/Modern-Web-Development-Beginner-s-Guide/>

但是请注意，本作品采用 [署名-非商业性使用 4.0 国际 (CC BY-NC 4.0)] 协议发布。
你可以自由转载、分发、分享，但必须注明作者及出处，不得用于任何商业用途。

否则在沟通无果后，Dontalk 团队不介意采用法律武器捍卫权益。

协议详情请见: <https://creativecommons.org/licenses/by-nc/4.0/deed.zh>

评价:

这份《现代网页开发入门指南书 v1.5》作为一份旨在引导初学者进入全栈Web开发的材料，展现了其覆盖广泛知识领域的雄心。它试图带领读者从后端的数据库设计，经过API开发，最终到前端的实现，并采用了当前较为流行的技术栈（如Node.js, Express, Sequelize, React, Vite）。文档通过代码示例来阐述概念，这对于初学者动手实践具有一定的积极意义。

然而，从技术严谨性和最佳实践指导的角度来看，这份指南在一些关键环节存在较为重要的改进空间(缺少动机阐述和不建议声明)(下面声明)，初学者若不加辨别地采纳，可能在实际项目中引入风险或不良实践。

值得肯定的方面：

1. **内容覆盖面广**：指南尝试覆盖Web开发的整个生命周期，从数据存储到用户交互，有助于初学者建立整体概念。
2. **技术栈选择现代**：采用了Node.js、Express、React、Vite等现代Web开发中常用的技术和工具，使学习者能够接触到当前主流的开发模式。
3. **实践性导向**：提供了大量的代码片段，涉及数据库模型定义、API路由、JWT认证、前端组件等，鼓励读者通过实践学习。
4. **核心概念涉及**：介绍了诸如ORM (Sequelize)、RESTful API设计、JWT认证机制、前端路由 (React Router)、组件化开发等Web开发的核心概念。
5. **安全意识初显**：在某些方面体现了对安全性的关注，例如提到了使用bcrypt 进行密码哈希，以及在JWT存储中讨论了 HttpOnly Cookie的应用。

现代网页开发入门指南书 v1.5 存在的问题

你可以粗略阅读此处，之后再回看

“现代网页开发入门指南书 - 从数据库设计，到后端，再到前端 v1.5版本.pdf”的摘录内容，我来分析其中可能存在的严重技术性问题，并提供相应的解读。

从整体上看，这份指南覆盖了从数据库设计到前后端实现的关键环节，并提供了一些实践代码。然而，在一些技术细节和最佳实践方面，确实存在一些可以被认为是“严重”或至少是“重要”的技术问题或改进点。

存在的一些问题(下文不好更改-此处声明(填坑))：

1. 数据库设计与 SQL 问题

- **问题 1.1: article 表中不必要的 article_id 字段 (用户已指出并解决)**
 - **原文描述**: 用户提供的 SQL 中 article 表同时有 id (主键) 和 article_id。
 - **严重性**: 中等。这会导致数据冗余和潜在的逻辑混乱。外键应该引用主键 id。
 - **状态**: 用户在后续的交流中已经指出了这个问题，并通过移除 article.article_id 并让 article_image.article_id 引用 article.id 来修正。
- **问题 1.2: 使用 CHAR(6) 作为自定义关联键 (Release level)**
 - **原文描述**: 提出使用名为 "Release level" (在 article 表) 和 "level" (在 article_image 表) 的 CHAR(6) 字段，通过随机6位数进行关联。
 - **严重性**: 中高。
 - **唯一性风险**: “随机6位数”如果生成算法不够健壮，有碰撞（重复）的风险，尤其是在数据量大时。UNIQUE 约束可以防止插入重复值，但不能解决生成算法本身的问题。

题。

- **性能:** 字符串类型的键通常比整数类型的键在索引和连接操作上性能稍差。
- **可读性和维护性:** 虽然在某些特定场景下自定义关联键有其用途, 但通常情况下, 使用自增整数主键 (id) 作为外键引用目标更简单、高效且符合常规实践。
- **SQL 字段名:** Release level 这样的带空格的字段名在 SQL 中需要用反引号 ``Release level`` 包裹, 容易出错且不推荐。
- **改进建议:** 坚持使用 `article.id` (主键) 和 `article_image.article_id` (外键) 进行关联。如果确实需要一个人类可读的、唯一的发布标识, 可以额外增加一个 `release_code` 字段, 并确保其唯一性, 但不一定用它作为主外键关联。
- **问题 1.3: ON UPDATE CASCADE 的普遍使用**
 - **原文描述:** 在外键定义中同时使用了 `ON DELETE CASCADE` 和 `ON UPDATE CASCADE`。
 - **严重性:** 低到中等。
 - `ON DELETE CASCADE` 是常见的, 表示删除主表记录时级联删除从表相关记录。
 - `ON UPDATE CASCADE` 表示更新主表主键值时, 级联更新从表外键值。虽然在某些数据库和场景下支持, 但主键本身通常是不应该被频繁更新的。如果主键是自增整数, 几乎永远不会更新。如果主键是可能变更的业务字段 (不推荐), `ON UPDATE CASCADE` 才显得更有意义。过度依赖此特性可能掩盖了主键设计不当的问题。
 - **改进建议:** 仔细评估是否真的需要 `ON UPDATE CASCADE`。对于自增主键, 它几乎没有作用。

2. Sequelize ORM 使用问题

- **问题 2.1: DataTypes.STRING(255) 的写法 (用户已提问)**
 - **原文描述:** 代码中使用了 `DataTypes.STRING(255)`。
 - **严重性:** 低。这不是一个“错误”, 但正如用户指出的, Sequelize v6+ 中 `DataTypes.STRING` 默认映射到数据库的 `VARCHAR(255)` (或数据库的默认字符串长度), 所以显式写 (255) 通常不是必需的, 除非有特定长度要求。
 - **状态:** 已在交流中澄清。
- **问题 2.2: timestamps: false 与手动 created_at**
 - **原文描述:** 模型定义中设置 `timestamps: false`, 然后手动定义 `created_at` 字段。
 - **严重性:** 低。这是一种可行的做法, 但 Sequelize 的 `timestamps: true` (默认) 会自动管理 `createdAt` 和 `updatedAt` 字段, 通常更方便。如果只需要 `createdAt`, 可以配置 `timestamps: true, updatedAt: false`。手动管理意味着在更新操作时也需要手动更新 `updatedAt` (如果需要的话)。
 - **改进建议:** 除非有特殊理由, 否则利用 Sequelize 内建的时间戳管理功能可能更简洁。
- **问题 2.3: sequelize.sync({ alter: true }) 的滥用**
 - **原文描述:** 在 `app.js` 或入口文件中使用 `await sequelize.sync({ alter: true })`; 来同步数据库。
 - **严重性:** 高 (在生产环境中)。

- `{ alter: true }` 会尝试修改现有表以匹配模型定义，这在开发初期可能很方便，但在生产环境中非常危险，可能导致数据丢失或表结构意外更改。
- `{ force: true }` (未在示例中直接出现，但与 `sync` 相关) 会先删除表再重建，同样会导致数据丢失。
- **改进建议:** 生产环境中应使用数据库迁移 (Migrations) 工具 (如 Sequelize CLI 提供的迁移功能) 来管理数据库模式的变更。开发环境中 `sync({ alter: true })` 可酌情使用，但需谨慎。
- **问题 2.4: 模型关联中 `sourceKey` 和 `targetKey` 的理解**
 - **原文描述:** 在 `Article.hasMany(ArticleImage, { foreignKey: 'article_id', sourceKey: 'id' })` 和 `ArticleImage.belongsTo(Article, { foreignKey: 'article_id', targetKey: 'id' })` 中，`sourceKey` 和 `targetKey` 的使用。
 - **严重性:** 低。这里的用法是正确的，因为它们都指向了 `Article` 表的 `id` 字段。
`sourceKey` 是源模型 (`Article`) 中与外键关联的键，`targetKey` 是目标模型 (`ArticleImage`) 中被外键引用的键。当外键引用的不是目标模型的主键时，或者源模型的关联键不是其主键时，这些选项才更有区分度。对于标准的主键-外键关联，有时可以省略。
 - **说明:** 当关联字段名与默认规则一致时 (例如，`ArticleImage` 有 `articleId` 字段引用 `Article` 的 `id`)，很多配置可以省略。显式配置更清晰，但需要确保理解每个选项的含义。

3. 后端 API 与安全问题 (基于 JWT 部分)

- **问题 3.1: JWT 密钥管理**
 - **原文描述:** `const JWT_SECRET = 'your_secret_key';`
 - **严重性:** 高 (如果直接用于生产)。
 - 密钥硬编码在代码中是非常不安全的。
 - **改进建议:** 密钥必须通过环境变量 (如 `.env` 文件配合 `dotenv` 库) 或安全的配置服务来管理，并且密钥本身应该是强随机字符串。
- **问题 3.2: 错误处理和信息泄露**
 - **原文描述:** 登录失败时返回 `res.status(404).json({ error: '用户未找到' });` 或 `res.status(401).json({ error: '无效的凭证' });`。
 - **严重性:** 中等。
 - 区分“用户未找到”和“密码错误”会给攻击者提供枚举有效用户名的信息。
 - **改进建议:** 对于登录失败，应返回统一的、模糊的错误信息，例如 `res.status(401).json({ error: '用户名或密码错误' });`。
- **问题 3.3: `HttpOnly` Cookie 的 `secure` 和 `sameSite` 属性**
 - **原文描述:** 提到了将 JWT 存储在 `HttpOnly` Cookie 中，并给出了配置示例。


```
res.cookie("authToken", token, {
  httpOnly: true, // 防止客户端JS访问
  secure: process.env.NODE_ENV === 'production', // 仅在HTTPS下传输
  maxAge: 3600000 * 1, // Cookie有效期 (例如1小时)
  sameSite: "Strict", // 防CSRF
});
```

- **严重性:** 低 (因为示例代码考虑到了这些)。这是一个好的实践。
- **关键点:** 确保 `secure: process.env.NODE_ENV === 'production'` 的逻辑正确, 并且生产环境强制 HTTPS。 `sameSite: "Strict"` 或 `"Lax"` 是防止 CSRF 的重要措施。
- **问题 3.4: 权限校验的粒度**
 - **原文描述:** `authorizeRole` 中间件示例 `if (!req.user || req.user.role !== requiredRole)`。
 - **严重性:** 中等 (取决于应用复杂度)。
 - 这种基于单一角色的校验对于简单应用尚可, 但复杂应用可能需要更细粒度的权限控制 (例如, 基于操作的权限, 或者用户同时拥有多个角色)。
 - **改进建议:** 考虑引入更完善的权限管理方案, 如 RBAC (Role-Based Access Control) 或 ABAC (Attribute-Based Access Control)。

4. 前端 React 与 Vite 问题

- **问题 4.1: API 地址硬编码**
 - **原文描述:** `fetch("http://localhost:3000/data")`
 - **严重性:** 中等 (在需要部署到不同环境时)。
 - API 地址硬编码不利于在不同环境 (开发、测试、生产) 中部署。
 - **改进建议:** 使用环境变量配置 API 基地址。例如, 在 Vite 中通过 `.env` 文件和 `import.meta.env.VITE_API_URL`。
- **问题 4.2: 错误处理和用户反馈**
 - **原文描述:** `catch (error) { console.error("Failed to fetch announcement:", error); setAnnouncement(" "); }`
 - **严重性:** 中等。
 - 仅在控制台打印错误对用户不友好。设置为空字符串可能也不是最佳的用户体验。
 - **改进建议:** 应该向用户显示明确的错误信息 (例如, 使用 Toast 通知、错误提示组件), 并考虑重试机制。
- **问题 4.3: 状态管理选择**
 - **原文描述:** 提到了 Context API 和 Redux/Zustand 等。
 - **严重性:** 低 (这是一个架构选择问题)。
 - **说明:** 指南中提到了多种状态管理方案是好的。选择哪种取决于应用规模和团队偏好。对于小型应用, `useState` 和 `useReducer + Context API` 可能足够。大型应用则可能

从 Redux、Zustand 等库中受益。

总结

指南中提供的内容基本覆盖了现代Web开发的一些核心概念和技术栈，并且在一些地方（如 HttpOnly Cookie 的使用）体现了较好的安全意识。

主要的“严重”技术性问题集中在：

1. 生产环境中 `sequelize.sync({ alter: true })` 的使用：这是最危险的一点，可能导致生产数据丢失。
2. JWT 密钥的硬编码：严重的安全隐患。
3. 登录错误信息过于具体：可能泄露用户信息。
4. 自定义关联键 `CHAR(6)` 的设计：相比标准整数主外键，风险更高，性能可能更差。

其他问题更多是关于“最佳实践”、“代码健壮性”和“可维护性”的改进点。这份指南作为一个“入门指南”，在简化概念的同时，也需要在关键的生产安全和最佳实践方面给予更明确的警告和指导。

数据库 - MariaDB

```
-- 用户管理表
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,           -- 用户唯一标识
  username VARCHAR(50) NOT NULL UNIQUE,        -- 用户名，唯一
  password VARCHAR(255) NOT NULL,              -- 用户密码（加密存储）
  email VARCHAR(100) NOT NULL UNIQUE,          -- 用户邮箱，唯一
  is_member TINYINT(1) DEFAULT 1,              -- 用户是否会员（1=激活，0=未激活）
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP, -- 创建时间
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
  CURRENT_TIMESTAMP -- 更新时间
);

-- 文章管理表
CREATE TABLE articles (
  id INT AUTO_INCREMENT PRIMARY KEY,           -- 文章唯一标识
  publisher_id INT NOT NULL,                   -- 发布文章的用户ID
  title VARCHAR(255) NOT NULL,                 -- 文章标题
  content TEXT NOT NULL,                       -- 文章内容
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP, -- 创建时间
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
  CURRENT_TIMESTAMP, -- 更新时间
);
```

```
FOREIGN KEY (id) REFERENCES users (id) ON DELETE CASCADE -- 用户删除时删除其文章
);

-- 文章图片表
CREATE TABLE article_images (
    id INT AUTO_INCREMENT PRIMARY KEY,          -- 图片唯一标识
    article_id INT NOT NULL,                     -- 关联文章的ID
    image_url VARCHAR(255) NOT NULL,             -- 图片路径或URL
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP, -- 创建时间
    FOREIGN KEY (article_id) REFERENCES articles (id) ON DELETE CASCADE -- 文章删除时级联删除图片
);
```

用户管理表的 最后更新时间

最后更新时间的意义和应用场景是什么呢？

意义

`updated_at` 字段的作用是记录一条数据的最后更新时间。它能够自动更新为数据被修改时的时间戳。这是一个非常实用的字段，尤其在需要追踪数据变化的场景中，可以轻松知道数据最近一次被操作的时间。

在设计数据库表结构中，`updated_at` 通常用于以下场景：

- **数据变更的记录：** 用来标记一条数据最后被修改的时间。
- **数据同步：** 在需要同步不同系统或服务器上的数据时，可以通过 `updated_at` 找到比某时间戳之后更新的数据。
- **调试和审计：** 跟踪数据的更新历史，快速定位问题或分析系统行为。

应用场景

在用户表中，`updated_at` 可用于记录用户信息的最后修改时间：

- **用户修改资料：** 当用户修改个人信息（如用户名、邮箱等）时，自动更新 `updated_at` 字段。
- **会员续费：** 当会员续费或到期时间更改时，`updated_at` 会更新为最新时间，方便追踪会员状态的变更。
- **调试问题：** 如果用户报告登录或账户问题，可以检查 `updated_at` 了解最近的变更时间，协助排查问题。

一般网站数据库如何存储图片呢？

方案 1：文件系统/云存储 + 数据库记录路径或 URL

- 优点：
 1. **性能更高：** 图片文件不存储在数据库中，数据库的读取和写入性能不会因图片数据增大。
 2. **易于扩展：** 图片可以通过 CDN 或云存储加速分发，提高访问速度。
 3. **维护简单：** 图片可以独立于数据库进行备份和管理。
 4. **存储成本低：** 文件存储（尤其是云存储）比数据库存储更加经济。
- 缺点：
 1. **数据一致性问题：** 图片文件和数据库记录可能不同步（如文件被意外删除，但数据库记录仍然存在）。
 2. **额外的存储配置：** 需要搭建文件服务器或对接云存储服务。

方案 2：将图片直接存储在数据库中

- 描述：

图片文件以二进制数据的形式存储在数据库中，使用 BLOB（Binary Large Object）或 TEXT 类型字段。
- 优点：
 1. **数据一致性：** 图片和其他文章数据存储在同一个数据库中，管理和备份更加方便。
 2. **易于迁移：** 数据库迁移时，图片数据会一并迁移，无需额外操作。
 3. **安全性更高：** 图片数据直接存储在数据库中，比文件系统更容易保护敏感信息。
- 缺点：
 1. **性能较差：** 数据库的读写性能可能会因存储大量图片而显著下降。
 2. **备份和恢复速度慢：** 数据库变得非常庞大后，备份和恢复会更加耗时。
 3. **存储成本高：** 数据库通常使用更昂贵的存储方案，存储大量图片会增加成本。

为什么不把图片放在和文章一个表里呢

**之后，用 逗号 隔开它们。

在一个表中保存多张图片是可以做到的，但这取决于实际需求和场景。如果直接在一个表的字段中保存多张图片路径（例如用英文逗号分隔），虽然实现简单，但在**扩展性、维护性和性能**方面存在一些限制，因此通常不推荐。

下面是详细的分析，为什么不建议在一个字段中保存多张图片路径，以及可以这样做的场景。

1. 数据库设计原则

- 数据库设计遵循 "第一范式 (1NF) " 的范式规则：
 - 每个字段应该只存储一个值，而不是一组值。
 - 如果一个字段存储了多个值（如用逗号分隔的图片路径），会违反第一范式，使得数据结构冗杂，难以管理和查询。

2. 查询复杂性

- 当数据存储为逗号分隔的形式时，查询和操作会变得困难。例如：
 - 查询某篇文章是否包含某张特定的图片路径需要用 LIKE 或 FIND_IN_SET，效率较低。
 - 统计或筛选包含特定图片的记录时，无法利用数据库的索引。
- 示例：查询包含 /uploads/images/image2.jpg 的文章

```
SELECT *  
FROM articles  
WHERE FIND_IN_SET('/uploads/images/image2.jpg', image_urls);
```

- 这种查询无法利用索引，性能较差。

3. 维护困难

- 当需要对单张图片进行操作时（比如删除、更新或替换图片路径），操作会变得复杂，容易出错。
- 例如，如果某张图片路径需要更新，必须解析整个字段并重新构建路径字符串。

示例：删除某张图片路径 - 如果字段 image_urls 的值为：

```
/uploads/images/image1.jpg,/uploads/images/image2.jpg,/uploads/images/image3.jpg
```

需要删除 /uploads/images/image2.jpg：

- 必须在应用层解析字符串，删除对应路径后再更新字段内容。
- 操作繁琐，容易遗漏或出错。

4. 扩展性不足

- 如果以后需要为每张图片添加额外信息（如图片的标题、描述、排序顺序等），逗号分隔的形式很难支持。

- 在单个字段下，图片的额外信息没法单独存储，无法满足复杂业务需求。

使用场景：

1. 图片数量较少

- 如果每篇文章的图片数量固定且较少（如 2-3 张），存储在一个字段中可能是可接受的。

2. 简单项目

- 在小型项目或实验性开发中，数据结构简单且无需复杂查询时，可以使用逗号分隔的形式存储图片路径。

3. 仅用于展示

- 如果图片仅用于简单展示，不需要复杂的查询、筛选或操作，逗号分隔的形式可以满足需求。

后端 - NodeJS

后端用 Express 制作 API 。Sequelize 操作 数据库。

项目目录结构：

```
├── app.js                # 整合文件
├── database              # 数据库-文件夹
│   └── database.js      # 数据库初始化
├── middlewares          # 中间件-文件夹
│   └── auth.js          # 认证中间件
├── models               # 数据库模型-文件夹
│   ├── Article.js       # 文章模型
│   ├── User.js          # 用户模型
│   └── index.js          # 合并索引
├── routes               # 路由-文件夹
│   ├── admin.js         # admin(后台)路由
│   └── login.js         # login(登录)路由
├── uploads              # 上传-文件夹
│   └── image             # 图片-文件夹
│       ├── "Title"      # 图片-分夹
│       ├── 12.jpg       # 图片
│       ├── 34.jpg       # 图片
│       ├── 56.jpg       # 图片
│       └── 78.jpg       # 图片
└── utils                # 工具-文件夹
    └── jwt.js            # JWT功能
```

使用上面的目录结构，使得项目更加清晰，利于维护。但是缺点也很明显，就是东西肉眼可见的多。

不喜欢这样，完全可以把 `middlewares` 和 `utils` 文件夹合并，`database` 和 `models` 文件夹合并。极端一点的话，可以把 `route` 写在一起，甚至是和 `app.js` 合并，把整个 `models` 和 `database` 合并为一。。。

缺点就是，不方便维护，你需要在一个文件里面找东西，改代码。会有一种牵一发而动全身的即视感。

不过也比所有东西写在一个 `app.js` 好。

代码&说明

1. Sequelize 操作数据库

1.1 安装 Sequelize 和相关依赖

在项目中安装 Sequelize 和数据库驱动（以 MySQL 为例）：

```
npm install sequelize mysql2
```

1.2 初始化 Sequelize

创建一个 `database.js` 文件，用于配置和初始化 Sequelize。

```
const { Sequelize } = require('sequelize');

// 创建 Sequelize 实例
const sequelize = new Sequelize('database_name', 'username', 'password', {
  host: 'localhost',           // 数据库地址
  dialect: 'mysql',           // 数据库类型 (mysql、postgres、sqlite 等)
  logging: false               // 禁用 SQL 查询日志 (可选)
});

// 测试数据库连接
(async () => {
  try {
    await sequelize.authenticate();
    console.log('Database connected successfully.');
```

```
    } catch (error) {
      console.error('Unable to connect to the database:', error);
    }
  })();
```

// 下面代码是同步数据库的代码，如果你的数据库表没有创建的话，使用下面代码，那么ORM库会自动帮你“同步（创建）”出你定义模型的表。

```
(async () => {
  try {
    await sequelize.sync({ alter: true }); // 根据模型更新表结构
    console.log('Database synchronized successfully.');
```

////////// 不过由于已经手动创建了表，并插入了测试数据，所以请忽略。 //////////

```
  } catch (error) {
    console.error('Error synchronizing database:', error);
  }
})();

module.exports = sequelize;
```

1.3 定义模型

在 `models/` 文件夹中创建模型文件，例如 `models/User.js` 和 `models/Article.js`。

先简单引用说明一下：

STRING 默认长度就是 255，所以不需要再写 (255)。INTEGER 亦不需要加 (255)，整数类型在 Sequelize 中不会受长度限制，MySQL 8.0 也已废弃显示宽度。

注意：DataTypes.INTEGER 接受的参数是 MySQL 的整数显示宽度（如 (12)），但 MySQL 8.0 及以上版本已经废弃了显示宽度，它不再影响存储或行为。所以下面，STRING等，之后不再需要(255)。

在 Sequelize 中，使用 `DataTypes.DATE` 表示 DATETIME 类型。

布尔值的默认行为：

```
is_active: {
  type: DataTypes.BOOLEAN,
  allowNull: false,
  defaultValue: true, // 默认值为 true
},
```

好了，我们切入正题

用户模型：User

值得注意的是，后台路由 `/admin` 没有加入判断管理员的逻辑，并且这个用户模型也没有明确区分出管理员和普通用户的列(字段)，所以 `/admin` 可以直接被该表的用户登陆。可能会对系统造成不可估量的损失。

解决方案：

1. 数据库

- 把在该表后面增加多一个列(字段)，来判断是否管理员。(推荐)
- 把管理员放在新的表。

2. 路由

- 在后台路由增加一个验证

(前端篇 后台路由 - 作进一步说明和解决)

```
const { DataTypes } = require('sequelize');
const sequelize = require('../database/database');

const User = sequelize.define('User', {
  id: {
    type: DataTypes.INTEGER,
    autoIncrement: true,
    primaryKey: true,
  },
  username: {
    type: DataTypes.STRING,
    allowNull: false,
    unique: true,
  },
  password: {
    type: DataTypes.STRING,
    allowNull: false,
  },
  email: {
    type: DataTypes.STRING,
    allowNull: false,
    unique: true,
  },
  // 如果填写不存在的列(因为没有使用sync同步)，所以会报错，
  // 导致查不出数据
  is_member: {
    type: DataTypes.BOOLEAN,
    allowNull: false,
    defaultValue: true,
  }
});
```

```

}, {
  timestamps: true, // 自动创建 createdAt 和 updatedAt 字段（由于没有书写
sync代码 所以需要手动在数据库里加入，之前加入的是created_at，所以需要改名，否则不认，报
错。再不然就false掉它（默认不填写 timestamps是true）
});

module.exports = User;

```

更改方式：

```

ALTER TABLE users
CHANGE COLUMN created_at createdAt datetime;

```

文章模型：Article

```

const { DataTypes } = require('sequelize');
const sequelize = require('../database/database');

const Article = sequelize.define('Article', {
  id: {
    type: DataTypes.INTEGER,
    autoIncrement: true,
    primaryKey: true,
  },
  title: {
    type: DataTypes.STRING,
    allowNull: false,
  },
  content: {
    type: DataTypes.TEXT,
    allowNull: false,
  },
  publisher_id: {
    type: DataTypes.INTEGER,
    allowNull: false,
  }
}, {
  timestamps: true,
});

module.exports = Article;

```

图片模型：ArticleImage

```

const { DataTypes } = require('sequelize');
const sequelize = require('../database/database');

const ArticleImage = sequelize.define('ArticleImage', {
  id: {
    type: DataTypes.INTEGER,
    autoIncrement: true,
    primaryKey: true,
  },
  article_id: {
    type: DataTypes.INTEGER,
    allowNull: false,
  },
  image_url: {
    type: DataTypes.STRING,
    allowNull: false,
  },
}, {
  timestamps: true, // 自动创建 createdAt 和 updatedAt 字段
});

module.exports = ArticleImage;

```

建立数据库关联 (v1.0版本 - 倾向理论)

(v1.3说: 非常建议读一下官方文档 - <https://sequelize.org/docs/v6/core-concepts/assocs/> (或者中文文档))

在 `models/index.js` 文件中定义模型间的关联关系，并且将它们一起暴露出去，给 `app.js` 使用。

我们先来说说背景，可以看到上面有两个模型，分别是 `User & Article` 它们分别对应了 用户模型 & 文章模型。那么我们如何将文章绑定给发布者呢？

文章模型(Article)处有一个列:叫做 `publisher_id`。而用户模型(User)处有一个列:叫做 `id`(也是该表的主键)。用户名，`email`不可重复，每个用户都是独一无二的(这不废话吗)，所以每个用户只有一个主键ID，这个ID也因为用户只可能在这个表出现一次，从而是唯一的。

那么这个ID就可以作为“快速识别码”，和文章模型(Article)的 `publisher_id` 相对应。比如说有10篇文章，`publisher_id` 都为1，在 `User` 这个表里面，1是管理员的ID。那显而易见，这十篇文章都是管理员发的。

我们要做的，就是在 `index.js` 整合它们，把它们一次性暴露给 `app.js`。虽然 `app.js` 也可以一个个导入 `Models` 文件夹里面的模型，但是这种面对大型项目情况下，就会显得力不从心。

```

const User = require('./User');
const Article = require('./Article');
const ArticleImage = require('./ArticleImage');

// 用户与文章：1对多关系
User.hasMany(Article, { foreignkey: 'publisher_id', onDelete: 'CASCADE' });
Article.belongsTo(User, { foreignkey: 'publisher_id' });

// 文章与图片：一对多关系
Article.hasMany(ArticleImage, {
  // 因为每个文章都是唯一的，图片应该和文章的 id 相对应，而非作者id。所以直接填写
  // article_id，直接使其匹配到 id 上。
  foreignkey: 'article_id',
  onDelete: 'CASCADE',
});
ArticleImage.belongsTo(Article, {
  foreignkey: 'article_id',
});

module.exports = { User, Article, ArticleImage };

```

或许你会有疑问：“users这个表里面没有publisher_id这一列啊，请问，上面代码的User的id是如何和publisher_id绑定在一起的呢？”

虽然 users 表中没有 publisher_id 列，但这并不会影响外键绑定，因为外键绑定的核心不是字段是否存在于主表（users 表），而是从属表（articles 表）中的外键字段如何引用主表的主键。

外键绑定的核心原理

在 Sequelize 中，外键绑定是通过以下逻辑实现的：

1. 主表（users）提供主键：

- users 表的主键是 id，这是默认的 Sequelize 主键字段。
- 主表（users）的主键将作为从属表（articles）中外键的目标。

2. 从属表（articles）包含外键列：

- 在 articles 表中有一个列 publisher_id，用于存储与 users 表 id 主键对应的值。
- articles.publisher_id 是从属表中的外键。

3. 关联定义：

- 在 Sequelize 中使用 foreignkey 指定从属表的外键字段名（publisher_id），并将其绑定到主表的主键（默认是 id）。
- 例如：

```
User.hasMany(Article, { foreignKey: 'publisher_id' });
Article.belongsTo(User, { foreignKey: 'publisher_id' });
```

代码解释:

```
User.hasMany(Article, { foreignKey: 'publisher_id', onDelete: 'CASCADE' });
Article.belongsTo(User, { foreignKey: 'publisher_id' });
```

- **User.hasMany(Article) :**
 - 告诉 Sequelize, User 表的主键 (id) 将被关联到 Article 表的外键 publisher_id 。
 - 每个 User 可以有多个 Article 。
- **Article.belongsTo(User) :**
 - 告诉 Sequelize, Article 表中的 publisher_id 字段引用了 User 表的主键 (id) 。
 - 每个 Article 只能属于一个 User 。
- 这种叫做「一对多关系」。

如果 User 模型没有 publisher_id 字段, 而 Article 模型(articles表)中有一个 publisher_id 外键列, Sequelize 会默认将 User 模型(users表)的主键 id 与 Article.publisher_id 进行关联。

CASCADE 是什么

这是个外键约束行为, onDelete: 'CASCADE' 表明了, 如果这个用户删除(从数据库), 那么另外一个表下面的, 有关他的所有文章, 都一并删除掉。

如果 users.userid 是随机生成的, 如何与 articles.id 绑定?

在这种情况下, 需要将 articles 表中的外键字段设置为 userid, 并通过 Sequelize 的 foreignKey 选项明确指定两者之间的关系。

我们来假设一下情况:

users 表: 主键是 userid, 随机生成。

userid (主键)	username
abcd1234	Alice
efgh5678	Bob

articles 表: 外键是 userid, 引用 users 表中的 userid 。

id (主键)	title	content	userid (外键)
1	First Article	Hello World	abcd1234
2	Second Post	More Content	abcd1234

Sequelize 模型定义：通过 `foreignKey` 明确指定外键字段名称。

```
const User = require('./User');
const Article = require('./Article');

// 用户与文章：一对多关系
User.hasMany(Article, {
  foreignKey: 'userid', // 指定外键为 articles 表中的 userid
  onDelete: 'CASCADE', // 如果用户被删除，相关文章也会被删除
});
Article.belongsTo(User, {
  foreignKey: 'userid', // 指定外键为 articles 表中的 userid
});
```

发现什么问题了吗？外键必须设置和受到关联那个表的列的列名一样，使其保持一致

在数据库设计中，外键的列名应与其所引用的主表的主键保持一致，或者至少让它们的逻辑含义明确对应。

建立数据库关联 (v1.3版本 - 独立的, 方便理解)

代码放在一个新的文件夹 - **Operational Database**

(非常建议读一下官方文档 - <https://sequelize.org/docs/v6/core-concepts/assocs/> (或者中文文档))

数据库表结构(SQL语句)与(目录结构):

我们有两个表，我们要对其进行关联。把表2(article_image) 关联给表1(article)

其中，表1(article)的表结构是：

Field	Type	Null	Key	Default	Extra

```

--+
| id          | int(10) unsigned | NO  | PRI | NULL          | auto_increment
|
| created_at  | timestamp        | NO  |     | current_timestamp() |
|
| title       | varchar(255)     | NO  |     | NULL           |
|
| content     | text             | NO  |     | NULL           |
|
+-----+-----+-----+-----+-----+
--+
4 rows in set (0.006 sec)

--- SQL 语句 ---
CREATE TABLE article (
    id INT(10) UNSIGNED AUTO_INCREMENT PRIMARY KEY, -- 主键，因为文章唯一，id也
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    title VARCHAR(255) NOT NULL,
    content TEXT NOT NULL
);

```

其次，表2(article_image)的表结构是：

```

+-----+-----+-----+-----+-----+
--+
| Field      | Type           | Null | Key | Default          | Extra
|
+-----+-----+-----+-----+-----+
--+
| id         | int(10) unsigned | NO  | PRI | NULL          | auto_increment
|
| created_at | timestamp        | NO  |     | current_timestamp() |
|
| article_id | int(10) unsigned | NO  | MUL | NULL           |
|
| image_url  | varchar(255)     | NO  |     | NULL           |
|
+-----+-----+-----+-----+-----+
--+
4 rows in set (0.006 sec)

--- SQL 语句 ---
CREATE TABLE article_image (
    id INT(10) UNSIGNED AUTO_INCREMENT PRIMARY KEY, -- 主键

```

```

    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    article_id INT(10) UNSIGNED NOT NULL,          -- 外键字段
    image_url VARCHAR(255) NOT NULL,
    FOREIGN KEY (article_id) REFERENCES article(id) -- 引用主键
    ON DELETE CASCADE
    ON UPDATE CASCADE
);

```

我们需要把 表1(article) 和 表2(article_image)关联在一起，方便我们查询。

目录结构：

```

project/
├── database/
│   └── database.js          // 数据库连接配置
├── models/
│   ├── Article.js          // Article 模型定义
│   ├── ArticleImage.js     // ArticleImage 模型定义
│   └── index.js             // 模型入口文件，统一导出
└── app.js                  // 主应用入口

```

把表1(article) - id(主键) 和 表2(article_image) - (article_id) 关联在一起。

代码编号为A1，存放在 Operational Database/A1

database/database.js:

负责配置并导出 Sequelize 实例。

```

const { Sequelize } = require('sequelize');

// 创建 Sequelize 实例
const sequelize = new Sequelize('数据库名', '数据库账号', '数据库密码', {
  host: 'localhost',
  dialect: 'mysql',
  logging: false, // 禁用 SQL 日志输出
});

// 测试数据库连接
(async () => {
  try {
    await sequelize.authenticate();
    console.log('Database connected successfully.');
```



```
    } catch (error) {
      console.error('Unable to connect to the database:', error);
    }
  })();

module.exports = sequelize;
```

models/Article.js:

定义 Article 模型。

```
const { DataTypes } = require('sequelize');
const sequelize = require('../database/database');

const Article = sequelize.define('Article', {
  id: {
    type: DataTypes.INTEGER.UNSIGNED,
    autoIncrement: true,
    primaryKey: true,
  },
  created_at: {
    type: DataTypes.DATE,
    defaultValue: DataTypes.NOW,
  },
  title: {
    type: DataTypes.STRING,
    allowNull: false,
  },
  content: {
    type: DataTypes.TEXT,
    allowNull: false,
  },
}, {
  tableName: 'article',
  timestamps: false, // 禁用自动生成的 timestamps 字段
});

module.exports = Article;
```

models/ArticleImage.js:

定义 ArticleImage 模型。

```
const { DataTypes } = require('sequelize');
const sequelize = require('../database/database');
```

```

const ArticleImage = sequelize.define('ArticleImage', {
  id: {
    type: DataTypes.INTEGER.UNSIGNED,
    autoIncrement: true,
    primaryKey: true,
  },
  created_at: {
    type: DataTypes.DATE,
    defaultValue: DataTypes.NOW,
  },
  article_id: {
    type: DataTypes.INTEGER.UNSIGNED,
    allowNull: false,
  },
  image_url: {
    type: DataTypes.STRING,
    allowNull: false,
  },
}, {
  tableName: 'article_image',
  timestamps: false, // 禁用自动生成的 timestamps 字段
});

module.exports = ArticleImage;

```

models/index.js:

统一导出所有模型，并设置关联关系。

```

const Article = require('./Article');
const ArticleImage = require('./ArticleImage');

// 定义关联关系
Article.hasMany(ArticleImage, {
  foreignKey: 'article_id', // 外键字段
  sourceKey: 'id',          // 主键字段
  onDelete: 'CASCADE',      // 设置级联删除
  onUpdate: 'CASCADE',      // 设置级联更新
});

ArticleImage.belongsTo(Article, {
  foreignKey: 'article_id', // 外键字段
  targetKey: 'id',          // 目标主键字段
});

module.exports = { Article, ArticleImage };

```

app.js:

主应用入口文件，调用模型并同步数据库。

```
const sequelize = require('./database/database');
const { Article, ArticleImage } = require('./models');

(async () => {
  try {
    // 同步数据库
    await sequelize.sync({ alter: true }); // `alter: true` 会更新表结构
    console.log('数据库同步成功');

    const newArticle = await Article.create({
      title: 'Test Article',
      content: 'This is a test article.',
    });

    await ArticleImage.create({
      article_id: newArticle.id,
      image_url: 'https://example.com/image1.jpg',
    });

    console.log('数据插入成功');

    // 查询测试
    const articles = await Article.findAll({
      include: [
        {
          model: ArticleImage,
          required: false, // 即使没有关联图片，也返回文章
        },
      ],
    });

    // 打印查询到的内容
    console.log(JSON.stringify(articles, null, 2));

  } catch (error) {
    console.error('数据库同步失败:', error);
  }
})();
```

返回数据:

```
[
  {
    "id": 1,
    "created_at": "2025-05-23T04:43:21.000Z",
    "title": "Test Article",
    "content": "This is a test article.",
    "ArticleImages": [
      {
        "id": 1,
        "created_at": "2025-05-23T04:43:21.000Z",
        "article_id": 1,
        "image_url": "https://example.com/image1.jpg"
      }
    ]
  }
]
```

获取文章及其图片

通过 `include` 选项查询 `Article` 及其关联的 `ArticleImage`。

```
const { Article, ArticleImage } = require('./models');

(async () => {
  try {
    const articles = await Article.findAll({
      include: [
        {
          model: ArticleImage,
          required: false, // 即使没有关联图片，也返回文章
        },
      ],
    });

    console.log(JSON.stringify(articles, null, 2));
  } catch (error) {
    console.error('Error fetching articles with images:', error);
  }
})();
```

查询某个特定文章及其关联的图片

通过 `where` 条件查询某篇特定文章及其关联的图片。

```
(async () => {
  const { Article, ArticleImage } = require('./models');

  try {
    const articleWithImages = await Article.findOne({
      where: { id: 1 }, // 查询 ID 为 1 的文章
      include: [
        {
          model: ArticleImage, // 包含其关联的图片
        },
      ],
    });

    console.log('Article with Images:',
      JSON.stringify(articleWithImages, null, 2));
  } catch (error) {
    console.error('Error fetching article by ID:', error);
  }
})();
```

查询某张图片及其对应的文章

通过 include 查询 ArticleImage 及其关联的 Article。

```
(async () => {
  const { Article, ArticleImage } = require('./models');

  try {
    const imageWithArticle = await ArticleImage.findOne({
      where: { id: 1 }, // 查询 ID 为 1 的图片
      include: [
        {
          model: Article, // 包含其关联的文章
        },
      ],
    });

    console.log('Image with Article:', JSON.stringify(imageWithArticle,
      null, 2));
  } catch (error) {
    console.error('Error fetching image by ID:', error);
  }
})();
```

核心步骤

1. 定义模型:

- **Article 模型:**

- 表名为 `article` , 包含字段 `id` (主键)、`created_at` (创建时间)、`title` (文章标题)、`content` (文章内容)。
- 禁用自动生成的时间戳字段 (`timestamps: false`)。

- **ArticleImage 模型:**

- 表名为 `article_image` , 包含字段 `id` (主键)、`created_at` (创建时间)、`article_id` (外键, 关联到 `Article` 的 `id`)、`image_url` (图片 URL)。
- 同样禁用自动生成时间戳字段。

2. 定义模型关联:

- `Article` 和 `ArticleImage` 通过 `article_id` 字段建立外键关联。
- **关系定义:**
 - `Article.hasMany(ArticleImage)` : 表示一个文章可以有多张图片。
 - `ArticleImage.belongsTo(Article)` : 表示每张图片属于一个文章。
- **设置外键约束:**
 - `onDelete: 'CASCADE'` : 当文章被删除时, 自动删除关联的图片。
 - `onUpdate: 'CASCADE'` : 当文章的主键被更新时, 自动更新关联的外键。

3. 同步数据库:

- 使用 `sequelize.sync({ alter: true })` 创建或更新数据库表结构。
- 确保模型定义和数据库结构保持一致。

4. 数据插入和查询:

- 插入一篇文章后, 插入与该文章关联的图片。
- 使用 `findAll` 方法查询文章及其关联的图片, 支持即使没有图片也返回文章的结果 (`required: false`)。

把表1(`article`) - `release_level`(主键) 和 表2(`article_image`) - (`release_level`) 关联在一起.

如果希望不是id, 而在两个表都有一个“Release level”的列, 通过里面的唯一值(随机6位数)来进行关联的话。

代码编号为A2, 存放在 **Operational Database/A2**

SQL语句 - 不要一次性全部执行(会报错)

```
DROP TABLE IF EXISTS article;

-- 创建 Article 表
CREATE TABLE article (
```

```

    id INT(10) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    title VARCHAR(255) NOT NULL,
    content TEXT NOT NULL,
    release_level CHAR(6) NOT NULL UNIQUE -- 唯一的 release_level 列
);

DROP TABLE IF EXISTS article_image;

-- 创建 ArticleImage 表
CREATE TABLE article_image (
    id INT(10) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    release_level CHAR(6) NOT NULL, -- 与 article.release_level 关联
    image_url VARCHAR(255) NOT NULL,
    FOREIGN KEY (release_level) REFERENCES article(release_level)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);

```

database/database.js - 不变

models/Article.js

```

const { DataTypes } = require('sequelize');
const sequelize = require('../database/database');

const Article = sequelize.define('Article', {
  id: {
    type: DataTypes.INTEGER.UNSIGNED,
    autoIncrement: true,
    primaryKey: true,
  },
  created_at: {
    type: DataTypes.DATE,
    defaultValue: DataTypes.NOW,
  },
  title: {
    type: DataTypes.STRING,
    allowNull: false,
  },
  content: {
    type: DataTypes.TEXT,
    allowNull: false,
  },
  release_level: {

```

```

        type: DataTypes.CHAR(6), // 固定6字符长度
        allowNull: false,
        unique: true, // 保证每个 release_level 唯一
    },
}, {
    tableName: 'article',
    timestamps: false,
});

module.exports = Article;

```

models/ArticleImage.js

```

const { DataTypes } = require('sequelize');
const sequelize = require('../database/database');

const ArticleImage = sequelize.define('ArticleImage', {
    id: {
        type: DataTypes.INTEGER.UNSIGNED,
        autoIncrement: true,
        primaryKey: true,
    },
    created_at: {
        type: DataTypes.DATE,
        defaultValue: DataTypes.NOW,
    },
    release_level: {
        type: DataTypes.CHAR(6), // 固定6字符长度
        allowNull: false,
    },
    image_url: {
        type: DataTypes.STRING,
        allowNull: false,
    },
}, {
    tableName: 'article_image',
    timestamps: false,
});

module.exports = ArticleImage;

```

models/index.js

```

const Article = require('./Article');
const ArticleImage = require('./ArticleImage');

```



```
// 定义关联关系
Article.hasMany(ArticleImage, {
  foreignKey: 'release_level', // 通过 release_level 进行关联
  sourceKey: 'release_level', // article 的 release_level
  onDelete: 'CASCADE',
  onUpdate: 'CASCADE',
});

ArticleImage.belongsTo(Article, {
  foreignKey: 'release_level', // article_image 的 release_level
  targetKey: 'release_level', // article 的 release_level
});

module.exports = { Article, ArticleImage };
```

app.js

```
const sequelize = require('./database/database');
const { Article, ArticleImage } = require('./models');

(async () => {
  try {
    await sequelize.sync({ alter: true }); // 自动同步表结构
    console.log('数据库同步成功');

    const releaseLevel = Math.random().toString(36).substring(2, 8).toUpperCase(); // 生成随机6位 release_level

    // 插入文章
    const newArticle = await Article.create({
      title: 'Test Article',
      content: 'This is a test article.',
      release_level: releaseLevel, // 插入随机 release_level
    });

    // 插入关联图片
    await ArticleImage.create({
      release_level: releaseLevel, // 使用相同的 release_level 进行关联
      image_url: 'https://example.com/image1.jpg',
    });

    console.log('数据插入成功');
  } catch (error) {
    console.error('数据库同步失败:', error);
  }
});
```

```
}  
})();
```

查询相关数据

```
const articlesWithImages = await Article.findAll({  
  include: [  
    {  
      model: ArticleImage,  
      required: false, // 即使没有关联图片也返回文章  
    },  
  ],  
});  
  
console.log(JSON.stringify(articlesWithImages, null, 2));
```

查询返回:

```
[  
  {  
    "id": 1,  
    "created_at": "2025-05-23T04:32:56.000Z",  
    "title": "Test Article",  
    "content": "This is a test article.",  
    "release_level": "V5D9SU",  
    "ArticleImages": [  
      {  
        "id": 1,  
        "created_at": "2025-05-23T04:32:56.000Z",  
        "release_level": "V5D9SU",  
        "image_url": "https://example.com/image1.jpg"  
      }  
    ]  
  }  
]
```

总结:

- `release_level` 是一个随机生成的 6 位唯一标识符, 用于关联 `article` 和 `article_image` 表。
- 在 `Sequelize` 中, 通过 `hasMany` 和 `belongsTo` 关联两个模型, 使用 `release_level` 作为外键字段。
- 外键的 `ON DELETE CASCADE` 和 `ON UPDATE CASCADE` 确保关联数据一致性。

把表1(article) - release_level(主键) 和 表2(article_image) - (level) 关联在一起.

假如Article表里面叫“Release level”，ArticleImage表里面叫“level”又如何关联

代码编号为A3，存放在 Operational Database/A3

SQL语句 - 不要一次性全部执行(会报错)

```
DROP TABLE IF EXISTS article;

-- 创建 Article 表
CREATE TABLE article (
  id INT(10) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  `Release level` CHAR(6) NOT NULL UNIQUE, -- 注意字段名带空格
  title VARCHAR(255) NOT NULL,
  content TEXT NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

DROP TABLE IF EXISTS article_image;

-- 创建 ArticleImage 表
CREATE TABLE article_image (
  id INT(10) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  level CHAR(6) NOT NULL, -- 关联到 article 的 `Release level`
  image_url VARCHAR(255) NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (level) REFERENCES article(`Release level`) -- 外键关联
  ON DELETE CASCADE
  ON UPDATE CASCADE
);
```

database/database.js: - 不变

models/Article.js

```
const { DataTypes } = require('sequelize');
const sequelize = require('../database/database');

const Article = sequelize.define('Article', {
  id: {
    type: DataTypes.INTEGER.UNSIGNED,
    autoIncrement: true,
    primaryKey: true,
```

```

    },
    releaseLevel: { // 映射到数据库中的 `Release level`
      type: DataTypes.CHAR(6),
      allowNull: false,
      unique: true,
      field: 'Release level', // 映射数据库字段名
    },
    title: {
      type: DataTypes.STRING,
      allowNull: false,
    },
    content: {
      type: DataTypes.TEXT,
      allowNull: false,
    },
  }, {
    tableName: 'article',
    timestamps: false,
  });

module.exports = Article;

```

models/ArticleImage.js

```

const { DataTypes } = require('sequelize');
const sequelize = require('../database/database');

const ArticleImage = sequelize.define('ArticleImage', {
  id: {
    type: DataTypes.INTEGER.UNSIGNED,
    autoIncrement: true,
    primaryKey: true,
  },
  level: { // 映射到数据库中的 `level`
    type: DataTypes.CHAR(6),
    allowNull: false,
  },
  imageUrl: {
    type: DataTypes.STRING,
    allowNull: false,
    field: 'image_url', // 映射数据库字段名
  },
}, {
  tableName: 'article_image',
  timestamps: false,
});

```

```
});  
  
module.exports = ArticleImage;
```

models/index.js

```
const Article = require('./Article');  
const ArticleImage = require('./ArticleImage');  
  
// 定义关联关系  
Article.hasMany(ArticleImage, {  
  foreignKey: 'level',          // ArticleImage 的外键  
  sourceKey: 'releaseLevel',    // Article 的目标键  
  onDelete: 'CASCADE',  
  onUpdate: 'CASCADE',  
});  
  
ArticleImage.belongsTo(Article, {  
  foreignKey: 'level',          // ArticleImage 的外键  
  targetKey: 'releaseLevel',    // Article 的目标键  
});  
  
module.exports = { Article, ArticleImage };
```

app.js

```
const sequelize = require('./database/database');  
const { Article, ArticleImage } = require('./models');  
  
(async () => {  
  try {  
    await sequelize.sync({ alter: true }); // 更新数据库表结构  
    console.log('数据库同步成功');  
  
    const releaseLevel = 'ABC123';  
  
    // 插入 Article 数据  
    const newArticle = await Article.create({  
      releaseLevel, // 自动映射到数据库的 `Release level`  
      title: 'Test Article',  
      content: 'This is a test article.',  
    });  
  
    // 插入关联的 ArticleImage 数据  
    await ArticleImage.create({
```

```

        level: releaseLevel, // 与 Article 的 releaseLevel 保持一致
        imageUrl: 'https://example.com/image1.jpg',
    });

    console.log('数据插入成功');

    } catch (error) {
        console.error('数据库同步失败:', error);
    }
    })();

```

查询关联数据

```

const articlesWithImages = await Article.findAll({
    include: [
        {
            model: ArticleImage,
            required: false, // 即使没有关联图片也返回文章
        },
    ],
});

console.log(JSON.stringify(articlesWithImages, null, 2));

```

返回数据：

```

[
  {
    "id": 1,
    "releaseLevel": "ABC123",
    "title": "Test Article",
    "content": "This is a test article.",
    "ArticleImages": [
      {
        "id": 1,
        "level": "ABC123",
        "imageUrl": "https://example.com/image1.jpg"
      }
    ]
  }
]

```

总结：

- 通过 `field` 属性, Sequelize 可以将模型字段与数据库中的字段名 (即带空格的或不同的字段名) 进行映射。
- 在模型关联中, 通过设置 `foreignKey` 和 `sourceKey` 或 `targetKey`, 可以灵活实现不同字段名的关联逻辑。
- 数据库中的 `Release level` 对应模型中的 `releaseLevel`, `level` 直接对应模型中的 `level`。

同步数据库

在主入口文件 (如 `app.js` 或 `index.js`) 中同步模型到数据库:

看是否需要, 在上面已经说过了, 这里只做(再次)展示

```
const sequelize = require('./database/database');
const { User, Article } = require('./models');

// 同步数据库
(async () => {
  try {
    await sequelize.sync({ alter: true }); // 根据模型更新表结构
    console.log('Database synchronized successfully.');
```

2. 添加 JWT 认证

2.1 安装 JWT 相关依赖

使用 `jsonwebtoken` 包进行 JWT 生成和验证:

```
npm install jsonwebtoken
```

2.2 配置 JWT

请注意，无论如何都不要泄漏密钥，密钥泄漏会导致 身份伪造，绕过，数据篡改，会话挟持等等问题！

在项目中创建一个 `utils/jwt.js` 文件，封装 JWT 的生成和验证逻辑。

```
const jwt = require('jsonwebtoken');

// 定义密钥和过期时间
const JWT_SECRET = 'your_secret_key'; // 密钥 - 这个不应该被外人知道，否则会被伪造 JWT
const JWT_EXPIRES_IN = '1h';          // Token 有效期

// 生成 JWT
const generateToken = (payload) => {
  return jwt.sign(payload, JWT_SECRET, { expiresIn: JWT_EXPIRES_IN });
};

// 验证 JWT
const verifyToken = (token) => {
  try {
    return jwt.verify(token, JWT_SECRET);
  } catch (error) {
    throw new Error('Invalid or expired token');
  }
};

module.exports = { generateToken, verifyToken };
```

2.3 创建认证中间件

在项目中创建一个 `middlewares/auth.js` 文件，定义 JWT 验证的中间件。

```
const { verifyToken } = require('../utils/jwt');

// 中间件：验证 JWT
const authenticateJWT = (req, res, next) => {
  const authHeader = req.headers.authorization;

  // 检查是否存在 Bearer Token
  if (authHeader && authHeader.startsWith('Bearer ')) {
    const token = authHeader.split(' ')[1];

    try {
```



```

const decoded = verifyToken(token); // 验证 JWT
req.user = decoded;                // 将用户信息附加到请求对象
next();                            // 放行(反弹回去路由代码那里)|记住, 后面
要考
} catch (err) {
  return res.status(401).json({ error: 'Unauthorized: Invalid token' });
}
} else {
  return res.status(401).json({ error: 'Unauthorized: No token provided'
});
}
};

module.exports = { authenticateJWT };

```

2.4 登录接口：生成 JWT

在路由文件(routes/login.js)中添加登录接口，用于生成 JWT。

注意：路由路径为 `/`，因为在 `app.js` 中，你已经将 `authRoutes` 挂载到 `/login`

```

const express = require('express');
const bcrypt = require('bcrypt');
const { User } = require('../models');
const { generateToken } = require('../utils/jwt');

const router = express.Router();

// 用户登录接口
router.post('/', async (req, res) => {
  // 注意!!! 这里路径为 '/', 因为在 整合文件 app.js 里面已经挂载到了login!
  const { username, password } = req.body;

  try {
    // 查找用户
    const user = await User.findOne({ where: { username } });

    if (!user) {
      return res.status(404).json({ error: 'User not found' });
    }

    // 验证密码
    const isPasswordValid = await bcrypt.compare(password, user.password);
    if (!isPasswordValid) {

```

```

    return res.status(401).json({ error: 'Invalid credentials' });
  }

  // 生成 JWT
  const token = generateToken({ id: user.id, username: user.username });

  res.json({ token });
} catch (error) {
  res.status(500).json({ error: 'Something went wrong' });
}
});

module.exports = router;

// 下面代码会发现，即便输入正确，但是都和数据库的密码匹配不上，因为每次的生成都不一样(加盐哈希)，所以不要手动加密再去比较。直接使用 bcrypt.compare 作比较就行。
// const saltRounds = 10; // 密码加盐次数
// const hashedPassword = await bcrypt.hash(password, saltRounds);
// const isValid = await bcrypt.compare(hashedPassword, user.password);
// 上面是错误的

// console.log(`输入的密码${password}`)
// console.log(`输入的密码哈希${hashedPassword}`)
// console.log(`数据库里的密码${user.password}`)

//下面是正确的
//const isValid = await bcrypt.compare(password, user.password);

```

2.5 保护 /admin 接口

在 /admin 接口中使用 JWT 中间件，确保只有经过认证的用户可以访问。

下面范例是，直接挂载到 /admin 这将意味着，你必须在 整合文件 app.js 挂到 /。(当然，挂载到其他地方也行，比如说 abc，那么情况就是： abc/admin)

```

const express = require('express');
const { authenticateJWT } = require('../middlewares/auth');

const router = express.Router();

// 受保护的 /admin 路由
router.get('/admin', authenticateJWT, (req, res) => {
  res.json({ message: 'Welcome to the admin panel', user: req.user });
});

```

```
});

module.exports = router;

// 上面的 authenticateJWT 的意思是，调用中间件，对应了 JWT 认证(auth.js)里面的
next(); 放行，返回到路由。
// 如果验证没有通过呢？当然就是不返回后面的 JSON - Welcome to the admin panel。
// 而是直接返回 auth.js 里面的错误信息 - 如：验证不通过/不合法
```

3. 主文件整合

在 app.js 或 index.js 中整合上述功能：

```
const express = require('express');
const bodyParser = require('body-parser');
const authRoutes = require('./routes/login');
const adminRoutes = require('./routes/admin');

const app = express();

// 中间件
app.use(bodyParser.json()); // JSON 化数据，和数据库返回数据做比较用

// 路由 | 这里的挂载仅作演示
app.use('/login', authRoutes); // 登录路由
app.use('/', adminRoutes); // 受保护路由

// 启动服务器
const PORT = 3000;
app.listen(PORT, async () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

登录认证

通过 PostMan 之类的程序进行测试，我们对 /login 路由进行登录测试看看。

这样做：POST - localhost:3000/login

之后在 Body 传入 raw - JSON

```
{
  "username": "admin",
  "password": "admin"
}
```

我已经在数据库手动插入了一个用户，账号密码都是 admin。

之后我们将包发出，就会返回一个 Token。

```
{
  "token":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Im5widXNlcm5hbm5kbiIsImIhdCI6MTc0NmZmZGMywzXhwIjozQ3MzYjczfQ.NuAY7IN50jDZWQxMeTxxxxxxxoRrc7gu5GAcc"
}
```

但是当我们再去访问 /admin 的时候，会发现未被授权，这是为什么呢。

是因为我们虽然得到了下发的 Token，但是我们没有加进去 Authorization，所以报错。平常的网站之所以不需要手动添加，是因为前端和浏览器帮你完成了这些工作。而你写的后端API，测试阶段，都需要自己手动加上(为了测试API)。或者你可以直接搓一个前端出来测试，不过一般都是写完后端，再写前端的。

我们在 Authorization 添加一个 Bearer Token，之后把 Token填进去就行：

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Im5widXNlcm5hbm5kbiIsImIhdCI6MTc0NmZmZGMywzXhwIjozQ3MzYjczfQ.NuAY7IN50jDZWQxMeTxxxxxxxoRrc7gu5GAcc
```

这个时候再 GET /admin 就一切大吉了。

```
{
  "message": "Welcome to the admin panel",
  "user": {
    "id": 1,
    "username": "admin",
    "iat": 174xxxx873,
    "exp": 174xxxx273
  }
}
```

暴露文章 - Title和Content可见，Images需登录

想要实现这个功能，游客可以看到文章 `title` 和 `content`，但 `images` 需要注册登录才能看到，可以通过后端的权限控制和前端的条件渲染来实现。

我们必须先温习一下 - 什么是中间件？

在 **Express.js** 中，中间件是一个函数，用于处理请求（`req`）和响应（`res`）之间的逻辑。它还可以通过调用 `next()` 将控制权传递给下一个中间件。

中间件的用途

1. 处理请求和响应：
 - 修改请求对象（`req`）或响应对象（`res`），例如解析请求体、添加用户信息等。
2. 执行逻辑：
 - 验证用户身份（如 `verifyToken`）。
 - 检查权限。
 - 日志记录。
3. 控制请求流程：
 - 决定请求是否继续（调用 `next()`）或终止（返回响应）。

中间件的意义

- **逻辑复用**：避免在每个路由中重复写相同的逻辑（如身份验证）。
- **代码清晰**：将复杂的功能分成小的、可维护的模块。
- **请求处理链**：通过多个中间件组合，形成灵活请求处理流程。

总结

用于处理请求、添加逻辑、控制流程，从而提高代码的复用性和可维护性。

1. 后端实现权限控制

示例：基于 JWT 或 Session 验证

在登录完成时，会下发一个JWT，登录成功后，可以返回 `username` 之类的信息给浏览器(前端程序)去携带，通过判断是否有这个头，就可以判断用户是否登录，如果登录再展示这些信息。

我们在 `middlewares/verify.js` 下面尝试读取用户是否有登录(是否有JWT, 有则分离)

```
// 复用 verifyToken 作为 Token 合法性校验和解密。
const { verifyToken } = require('../utils/jwt');

function verifyLogin(req, res, next) {
  const authHeader = req.headers.authorization;
  // 如果没有 Authorization 请求头, 则视为未登录

  if (!authHeader) {
    console.log('No Authorization header found');
    req.user = null;
    return next();
  }

  // 提取 token (处理 Bearer 前缀)
  const token = authHeader.startsWith('Bearer ') ? authHeader.split(' ')[1] : authHeader;

  try {
    // 验证并解码 token
    const decoded = verifyToken(token);
    console.log('Decoded Token:', decoded);

    // 检查解码后的信息是否完整
    if (!decoded || !decoded.username) {
      console.error('Decoded token missing required fields:', decoded);
      req.user = null;
      return next();
    }

    req.user = decoded; // 将解码后的用户信息传递到后续中间件
    next();
  } catch (err) {
    console.error('Token verification failed:', err.message); // 打印具体错误信息
    req.user = null;
    next();
  }
}

module.exports = { verifyLogin };
```

routes/article.js 是无法复用之前的 verifyToken 中间件的，为什么？因为你会因为没有 Token 而被拦截，不放行。因为那个中间件是用来判断你是否登录，有无有效 Token，否则不允许你访问被保护的 API。如后台

```
const express = require('express');
const { verifyLogin } = require('../middlewares/verify'); // 验证登录中间件
const { Article } = require('../models')

const router = express.Router();

// 获取文章详情
router.get('/articles/:id', verifyLogin, async (req, res) => {
  const { id } = req.params;
  const user = req.user; // 登录信息通过 中间件 解析到 req.user

  try {
    const article = await Article.findById(id, {
      attributes: user
        ? ['title', 'content', 'images'] // 登录用户可以看到所有字段 | 如果列(字段)
        : ['title', 'content'], // 游客只能看到 title 和 content
    });
    // 不存在, 会报500

    if (!article) {
      return res.status(404).json({ message: 'Article not found' });
    }

    res.json(article);
  } catch (err) {
    res.status(500).json({ message: 'Internal server error' });
  }
});

module.exports = router;
```

之后我们需要更新一下 app.js

```
const express = require('express');
const bodyParser = require('body-parser');
const authRoutes = require('./routes/login');
const adminRoutes = require('./routes/admin');
const articleRoutes = require('./routes/article') // 添加这行

const app = express();
```

```
// 中间件
app.use(bodyParser.json());

// 路由
app.use('/login', authRoutes); // 登录路由
app.use('/', adminRoutes);    // 受保护路由

app.use('/', articleRoutes)    // 添加这行

// 启动服务器
const PORT = 3000;
app.listen(PORT, async () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

假如 Article 模型里对应的 articles 库存在一条数据：

```
id: 1
title: hello
content: neo
images: /data/xxx/1.jpg
```

之后请求 GET /article/1 ，如果没登录，那么它应该返回

```
{
  "title": "hello",
  "content": "neo"
}
```

带上 Token 去 GET，则返回

```
{
  "title": "hello",
  "content": "neo",
  "images": "/data/xxx/1.jpg"
}
```

2. 前端实现条件渲染

示例：React

API请求函数

```
async function fetchArticle(id, token = null) {  
  const headers = token ? { Authorization: `Bearer ${token}` } : {};  
  const response = await fetch(`/api/articles/${id}`, { headers });  
  return response.json();  
}
```

React组件

```
import React, { useEffect, useState } from 'react';  
  
function ArticlePage({ articleId, isLoggedIn, token }) {  
  const [article, setArticle] = useState(null);  
  
  useEffect(() => {  
    fetchArticle(articleId, isLoggedIn ? token : null).then(setArticle);  
  }, [articleId, isLoggedIn, token]);  
  
  if (!article) {  
    return <div>Loading ... </div>;  
  }  
  
  return (  
    <div>  
      <h1>{article.title}</h1>  
      <p>{article.content}</p>  
      {isLoggedIn && article.images && (  
        <div>  
          <h2>Images:</h2>  
          <img src={article.images} alt="Article" />  
        </div>  
      )}  
      {!isLoggedIn && <p>Login to see the images.</p>}  
    </div>  
  );  
}  
  
export default ArticlePage;
```

不过最好不要前端去做，因为抓包等一些原因，有可能会流出来。在一开始，也就是API，不要放出数据，更稳妥——也就是通过中间件。

API 加入注册和注册码校验

我们先创建两个文件，一个是 `models/ActivationCode.js`，一个是 `routes/register.js`。一个数据库模型，一个注册路由。

以及需要在 `models/index.js` 追加内容，以暴露 `ActivationCode` 数据库模型。再在 `app.js` 挂载 `routes/register.js` 路由。

数据库模型 `models/ActivationCode.js`

```
const { DataTypes } = require('sequelize');
const sequelize = require('../database/database');

const ActivationCode = sequelize.define('activation_code', {
  id: {
    type: DataTypes.INTEGER,
    autoIncrement: true,
    primaryKey: true,
  },
  code: {
    type: DataTypes.STRING,
    allowNull: false,
    unique: true,
  },
  valid_day: {
    type: DataTypes.INTEGER,
    allowNull: false,
  },
  is_permanent: {
    type: DataTypes.BOOLEAN,
    allowNull: true,
    defaultValue: false,
  },
  is_used: {
    type: DataTypes.BOOLEAN,
    allowNull: true,
  },
  user_by: {
    type: DataTypes.INTEGER,
    allowNull: true,
  },
}, {
  timestamps: true,
});
```

```
module.exports = ActivationCode;
```

关于 数据表模型:

- 邀请码数据模型 `user_by` 通过 用户ID 来标记。
- `is_used` 的 1/0 用来表示 注册码 使用与否

路由文件 `routes/register.js`

```
const express = require('express');
const { ActivationCode, User } = require('../models');
const bcrypt = require('bcrypt'); // 用于加密密码
const { Op } = require('sequelize'); // 引入 Sequelize 运算符

const router = express.Router();

router.post('/', async (req, res) => {
  const { username, password, email, code } = req.body;

  // 检查是否提供了激活码
  if (!code) {
    return res.status(400).json({ message: "The invitation code is empty." });
  }

  // 下面提供了两个思路
  // 用户名 直接用递交过来的 username(用户名) 为条件查询, 看看是否存在。
  // 激活码 先查询所有的激活码, 之后与递交过来的 code(激活码) 做匹配。

  try {
    // 检查用户名或邮箱是否已存在
    const existingUser = await User.findOne({
      where: {
        [Op.or]: [
          { username: username },
          { email: email }
        ]
      }
    });

    // 如果用户已存在
    if (existingUser) {
      return res.status(400).json({ message: 'Username or email'
件
```

```
already exists' });
};

// 查询所有未使用的激活码
const aCode = await ActivationCode.findAll({
  attributes: ['code'],
  where: { is_used: 0 },
});

// 提取激活码列表
const codes = aCode.map(item => item.code);

// 检查提供的激活码是否在未使用的激活码列表中
if (codes.includes(code)) {
  // 匹配成功，返回成功响应
  // return res.json({ message: 'DONE' });

  // 匹配成功，激活码有效，保存用户信息
  const hashedPassword = await bcrypt.hash(password, 10); // 加密密码

  const newUser = await User.create({
    username,
    password: hashedPassword, // 存储加密后的密码
    email,
  });

  // 更新激活码状态为已使用
  await ActivationCode.update(
    { is_used: 1 }, // 设置为已使用
    { where: { code: code } }
  );

  return res.json({
    message: 'Successful registration!',
    user: { // 返回新用户的相关信息，强化客户端体验
      id: newUser.id,
      username: newUser.username,
      email: newUser.email,
    },
  });
} else {
  // 匹配失败，返回错误信息
  return res.status(404).json({ message: "The invitation code is invalid or used." });
}
```

码

```

    }
  } catch (error) {
    console.error(error);
    res.status(500).json({ error: 'Server Error' });
  }
});

module.exports = router;

```

关于代码的话，上面的注释已经够清楚，提供了两个判断思路，但是我认为前者更加合适。

需要注意的是，在逻辑完成后，如果没有状态码(或直接return)，而直接 send json 回去的话，代码会继续执行下去的。所以一些 if 完毕后，要返回状态码。不要只是 send。

但是如果没有任何 send 返回，也会导致测试工具(页面)一直处于加载状态。所以在代码正确地走下去(一路畅通 if)的情况下，也要记得 send / return 数据回去。

对于 非json对象，比如说一些普通的“通告”，要使用 message 。如：res.json({ message: 'Successful.' });

绑定事务：

我认为这已经不算是 注意 了，应该是 **！警告！**

为什么需要警告？因为在创建用户和更新激活码的操作是独立的，如果在保存用户成功后，更新激活码状态失败，会导致激活码仍然可用，但用户已经注册成功。这种情况会产生数据不一致的问题。

使用 Sequelize 的事务（transaction）确保两个操作要么同时成功，要么同时失败。

路由文件 routes/register.js - 带绑定事务(这样才对)

```

// 追加内容
const transaction = await sequelize.transaction(); // 开启事务

try {
  // 保存用户信息
  const newUser = await User.create({
    username,
    password: hashedPassword,
    email,
  }, { transaction });

  // 更新激活码状态
  await ActivationCode.update(

```

```

    { is_used: 1 },
    { where: { code: code }, transaction }
  );

  await transaction.commit(); // 提交事务
  return res.status(201).json({ message: 'User created successfully',
user: newUser });
} catch (error) {
  await transaction.rollback(); // 回滚事务
  console.error(error);
  return res.status(500).json({ error: 'Server Error' });
}

```

整体代码就是：

```

const express = require('express');
const { ActivationCode, User, sequelize } = require('../models');
const bcrypt = require('bcrypt');
const { Op } = require('sequelize');

const router = express.Router();

router.post('/', async (req, res) => {
  const { username, password, email, code } = req.body;

  if (!code) {
    return res.status(400).json({ message: "The invitation code is empty." });
  }

  const transaction = await sequelize.transaction();

  try {
    // 检查用户名或邮箱是否已存在
    const existingUser = await User.findOne({
      where: {
        [Op.or]: [
          { username: username },
          { email: email }
        ]
      }
    });

    if (existingUser) {
      return res.status(409).json({ message: 'Username or email

```

```
already exists' });
}

// 检查激活码是否有效
const validCode = await ActivationCode.findOne({
  where: { code: code, is_used: 0 }
});

if (!validCode) {
  return res.status(400).json({ message: "The invitation code is invalid or used." });
}

// 验证密码强度
if (password.length < 8) {
  return res.status(400).json({ message: 'Password must be at least 8 characters long' });
}

// 加密密码
const hashedPassword = await bcrypt.hash(password, 10);

// 创建用户
const newUser = await User.create({
  username,
  password: hashedPassword,
  email,
}, { transaction });

// 更新激活码状态
await ActivationCode.update(
  { is_used: 1 },
  { where: { code: code }, transaction }
);

await transaction.commit(); // 提交事务
res.status(201).json({
  message: 'User created successfully',
  user: {
    id: newUser.id,
    username: newUser.username,
    email: newUser.email,
  }
});
} catch (error) {
  await transaction.rollback(); // 回滚事务
```

```
        console.error(error);
        res.status(500).json({ error: 'Server Error' });
    }
});

module.exports = router;
```

之后我们需要在 `models/index.js` 和 `app.js` 追加些东西

追加模型暴露 `models/index.js`

```
const User = require('./User');
const Article = require('./Article');
const ArticleImage = require('./ArticleImage');
const ActivationCode = require('./ActivationCode'); // 后添加

// 用户与文章：一对多关系
User.hasMany(Article, {
  foreignKey: 'publisher_id',
  onDelete: 'CASCADE',
});
Article.belongsTo(User, {
  foreignKey: 'publisher_id',
});

// 文章与图片：一对多关系
Article.hasMany(ArticleImage, {
  foreignKey: 'article_id', // 因为每个文章都是唯一的，图片应该和文章的 id 相对应，而非作者id。所以直接填写 article_id，直接使其匹配到 id 上。
  onDelete: 'CASCADE',
});
ArticleImage.belongsTo(Article, {
  foreignKey: 'article_id',
});

// 后添加 - 激活码功能 // user_by 与 主键ID 绑定
User.hasMany(ActivationCode, {
  foreignKey: 'user_by',
  onDelete: 'SET NULL',
});
ActivationCode.belongsTo(User, {
  foreignKey: 'user_by',
});
```



```
module.exports = { User, Article, ArticleImage, ActivationCode }; // 追加暴露
```

追加路由挂载 app.js

```
const express = require('express');
const bodyParser = require('body-parser');
const authRoutes = require('./routes/login');
const adminRoutes = require('./routes/admin');
const articleRoutes = require('./routes/article') // 添加这行
const app = express();

// 中间件
app.use(bodyParser.json());

// 路由
app.use('/login', authRoutes); // 登录路由
app.use('/', adminRoutes); // 受保护路由
app.use('/', articleRoutes) // 添加这行

// 启动服务器
const PORT = 3000;
app.listen(PORT, async () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

indexRoutes - 主页路由

之后我们写一个主页路由(routes/index.js)，显示最近 5 条最新的数据出来，供游客无偿浏览。只提供核心代码，来试试手：

```
.....
.....

try {
  const indexData = await Article.findAll({
    order: [['id', 'DESC']], // 按 id 降序排列
    limit: 5, // 限制为 10 条数据
  });

  if(!indexData) {
    return res.status(404).json({ message: 'Article not found' });
  };
};
```

```
res.json(indexData);
```

```
...  
.....  
.....
```

网站基本信息 - 通过读取JSON返回

这里提供一个 读取 JSON 文件来返回 网站详细 信息的方案，如 "标题", "副标题", "公告" 等等..

1. 我们接着在 models 文件夹创建一个文件，如 models/siteintro.js
2. 之后创建一个文件夹用来存放 JSON，如 config/SiteIntro.json

models/siteintro.js 文件内容

```
const fs = require('fs').promises; // 读取 JSON 文件(异步)  
const path = require('path'); // 可有可无，只是方便管理 JSON 路径  
const express = require('express');  
  
const router = express.Router();  
  
// 使用 path 管理 JSON 文件路径  
const jsonFilePath = path.join(__dirname, '../config/SiteIntro.json');  
  
router.get('/', async (req, res) => {  
  try {  
    const data = await fs.readFile(jsonFilePath, 'utf8'); // 读取文件内容  
    const readData = JSON.parse(data); // 解析 JSON  
    res.send({ readData }); // 返回解析后的数据  
  } catch (error) {  
    console.error(error);  
    res.status(500).json({ message: 'Internal server error' });  
  }  
});  
  
module.exports = router;
```

config/SiteIntro.json 文件内容

```
{
  "title": "xxxx",
  "subtitle": "xxxxxx",
  "announcement": "Welcome"
}
```

之后在 `app.js` 进行挂载

```
const express = require('express');
const bodyParser = require('body-parser');
const indexData = require('./routes/siteintro') // 追加

...

const cors = require("cors");

...

// 全局启用 CORS
app.use(cors());

// 路由
app.use('/', indexRoutes);
app.use('/data', indexData); // 追加
app.use('/article', articleRoutes);
...
```

之后访问接口 `/data` 就会返回数据

```
{
  "readData": {
    "title": "xxxx",
    "subtitle": "xxxxxx",
    "announcement": "Welcome"
  }
}
```

之后前端 `fetch` 后就可以直接剥洋葱式使用了。

CORS - 无法获取 API 数据问题

或许你在对接前端的时候，会发现访问 API 不被授权/不安全。这是因为 CORS 跨域问题，浏览器把你的流量截断了，通过这种方式可以放行所有的接口。

安装 cors

```
npm install cors
```

往 App.js 内追加代码

```
const express = require('express');
const bodyParser = require('body-parser');

...

const cors = require("cors");      // 追加

const app = express();

...

// 全局启用 CORS
app.use(cors());      // 追加

// 路由
app.use('/', indexRoutes);
...
```

即可解决。也可以按照文档，自己选择允许哪些 IP/地址 访问。

前端 - React

本文对 React 基本语法不作任何解释，如幽灵标签与闭合等等。。只解释疑难杂症和项目上手

前端使用 React 挂载数据。使用 Tailwind CSS 框架。

创建项目 - 官方(不推荐)

先不要跟着做

```
> npx create-next-app@latest
```

Need to **install** the following packages:

create-next-app@15.3.2

Ok to proceed? (y) y

```
✓ What is your project named? ... xxxxx
✓ Would you like to use TypeScript? ... No ☒ / Yes
✓ Would you like to use ESLint? ... No ☒ / Yes
✓ Would you like to use Tailwind CSS? ... No / Yes ☒
✓ Would you like your code inside a `src/` directory? ... No / Yes ☒
✓ Would you like to use App Router? (recommended) ... No / Yes ☒
✓ Would you like to use Turbopack for `next dev`? ... No ☒ / Yes
✓ Would you like to customize the import alias (`@/*` by default)? ... No ☒ /
Yes
.....
...
```

项目创建完毕后，我们打开 `package.json` 文件，可以看到下面内容

```
{
  "name": "项目名字",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "PORT=3005 next dev", // 更改这一行。在前面添加 PORT=3005
    "build": "next build",
    "start": "next start",
    "lint": "next lint"
  },
  "dependencies": {
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "next": "15.3.2"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "tailwindcss": "^4"
  }
}
```

我们需要按照上面一样更改，因为 `3000` 端口在本地开发环境里被 `Express` 占用了，所以我们就用 `3005` 端口吧

之后进入目录，启动项目就行

```
npm run dev
```

项目目录结构：

```
├── .
├── XXXX                                # 项目名
├── README.md                          # 项目详情
├── src/app                            # 核心目录，用于存放页面、布局 and 全局样式等内容
│   ├── favicon.ico
│   ├── globals.css
│   ├── layout.js
│   └── page.js
├── jsconfig.json
├── next.config.mjs
├── package-lock.json
├── package.json
├── postcss.config.mjs
└── public                            # 存放静态资源（如图片、字体等）此目录文件，编译时不会重命名
    ├── file.svg
    ├── globe.svg
    ├── next.svg
    ├── vercel.svg
    └── window.svg
```

可以看到，项目目录有点云里雾里的，有些东西不知道如何存放才好。

创建项目 - Vite(推荐) - 支持各种前端框架创建

如果你无意参加编程语言的派别之争，那么使用 **Vite** 去创建 **React** 无疑是非常好的选择，尤其面对新手。

兼容性说明：

Vite 需要 版本 18+ 或 20+。但是，某些模板需要更高的 Node.js 版本才能运行，如果您的包管理器出现警告，请升级。

Vite 官网: <https://vite.dev/guide/>

```
> npm create vite@latest
Need to install the following packages:
```

```
create-vite@6.5.0
Ok to proceed? (y) y
```

```
> npx
```

```
> create-vite
```

```
|
◇ Project name:
|   XXXX
|
◇ Select a framework:
|   React
|
◇ Select a variant:
|   JavaScript + SWC
|
◇ Scaffolding project in /path/to/project...
|
└ Done. Now run:
```

```
cd XXXX
```

```
npm install
```

```
npm run dev
```

项目目录结构

看起来好多了

```
.
├── XXXX
│   ├── README.md
│   ├── eslint.config.js
│   ├── index.html          # index 躯壳页面 - jsx最终挂载上这里(main.jsx)
│   ├── package.json
│   └── public              # 存放静态资源（如图片、字体等）此目录文件，编译时不会重命名
│
├── vite.svg                # 非 require 方式引入(如src)的内容(如LOGO)，不可以变更名字
│
├── src                    # 源码文件
│   ├── App.css           # 对应了 App 这个页面的CSS(但ReactCSS不是你想那么简单)
│   └── App.jsx            # 代码的根，根路由，它提供给了 main.jsx 挂载 - 整理路由和逻辑用
│
├── assets                 # 资源地址(这里的文件编译时会被重命名)
└── react.svg
```

```
|   |— index.css # main.jsx 的CSS文件
|   |— main.jsx  # 挂载文件 - 挂载上index.html - (真正的)index 文件
|— vite.config.js
```

5 directories, 11 files

main.jsx 和 App.jsx 的关系 (重要)

说实话，这两个真的把人整得云里雾里的，这里说明一下。

main.jsx - 项目的入口文件

main.jsx 是项目的入口文件？很奇怪对吧，毕竟写惯了程序，入口文件不应该给定参数，调取参数吗？事实上，我也被这个概念搞得晕头转向。所以，你只需要知道，**这个文件是最终挂载，渲染根路由(App.jsx)，或者更多的大组件到 index.html 就行**

为什么要说 大组件 ...我都要哭了，因为那些路由之类的处理，一般都是 App.jsx 这个根组件完成就行。除非你的项目足够大型和复杂，不然一个 App.jsx，之后 App.jsx 递交给 main.jsx 挂载到 index.html 就行了。

main.jsx - 它负责挂载 App.jsx 到 HTML 的根节点 (index.html)

- 作用：
 - 初始化 React 应用。
 - 将 React 应用挂载到 HTML 中的根节点 (通常是 `<div id="root">`)。
 - 引入全局样式文件 (如 `index.css`)。
 - 渲染整个 React 应用的根组件 (App)。

核心职责：

- 负责启动 React 应用：
 - 使用 `ReactDOM.createRoot()` 创建 React 的根节点。
 - 使用 `.render()` 方法，将 React 的根组件 (App) 渲染到页面中。
- 引入全局资源：
 - 加载全局样式 (如 `index.css`)。
 - 加载应用的根组件 (App.jsx)。

App.jsx - React 应用的根组件

- 作用：
 - 定义整个应用的核心逻辑和页面结构。
 - 包含路由配置 (如 React Router 的 `Routes` 和 `Route`)。

- 组织并引入其他子组件（如 Header、Footer、Home 等）。
- 可以包含全局状态管理（如 Context API 或 Redux 的 Provider）。

核心职责：

- 负责布局和页面逻辑：
 - 定义页面的整体结构（如头部、导航、主内容区）。
 - 配置页面路由，加载不同的页面组件。
- 组织组件：
 - 引入其他子组件，并将它们组合起来构建应用的 UI。
- 控制全局状态：
 - 如果使用 Context API、Redux 等全局状态管理工具，通常会在 App.jsx 中初始化状态管理。

我们进入目录，执行下面命令即可启动 React 测试服务器

```
npm install
npm run dev    ## 不需要额外更改端口，因为vite默认使用 5173 这个端口 - （自带热更新）
```

删除一些东西

你可以启动服务器，并且熟悉一下项目目录。但是出于开发一个新的页面目标出发，我们需要删除一些东西。

```
.
├── XXXX
│   ├── README.md
│   ├── eslint.config.js
│   ├── index.html          # 里面一些东西可以看着修改(如Title)
│   ├── package.json
│   └── public              # 存放静态资源（如图片、字体等）此目录文件，编译时不会重命名
├── vite.svg                # LOGO，删除即可
├── src                    # 源码文件
│   ├── App.css            # 删除文件里面的代码
│   ├── App.jsx            # 删除 return 里面的代码(以及App函数里面一行const)
│   ├── assets             # 资源地址(这里的文件编译时会被重命名)
│   │   └── react.svg      # react图标，删除即可
│   ├── index.css          # 删除文件里面的代码
│   └── main.jsx            # 无需删除任何，但是之后会作改变
```

```
└─ vite.config.js
```

5 directories, 11 files

安装React-Router(路由)

Vite 创建项目时，一般不会询问你是否引入 **React-Router** 功能，所以我们需要自己安装

我们需要使用到 **页面路由** (现代网页开发就几乎没有不用的)，所以需要安装：

```
npm install react-router-dom
```

页面路由，比如说在页面快速切换 -主页 -关于我们 -我的 功能页，而不重新刷新网页。

基本语法和项目文件增减在后面安装完 **CSS** 框架后说

不使用CSS框架 / 自己写一些CSS (警告⚠)

为什么是重要呢，因为 **React** 导入的 **CSS 是全局的**。这意味着，你在一个 `pages/xxx.jsx` 导入的 **CSS 文件**，将会被全局应用到 `App.jsx`, `main.jsx` 等等文件，一旦出现 **选择器(class等)** 名称相同，就会被 **CSS规则命中**。

所以你必须(最好)这样做：

1. 创建 **CSS 文件**时，不要使用 `.css` 结尾，要使用 `.module.css` 结尾(或者 `.module.scss`)
2. 在 **jsx 源码文件**，这样导入 **CSS**： `import importStyles from './xxxx.module.css'`;
3. 在**代码中**，这样命名元素： `<button className={importStyles.button}>`
这就是最原汁原味的用法。

当然你也可以使用 **CSS-in-JS**，安装库： `npm install styled-components`

如果，你在该页面下，没有用到多个 **CSS 文件**，而且 **ClassName** 没有重合，可以直接这样导入：

```
import './xxxx.module.css';
```

CSS框架(TailwindCSS / Bootstrap5)

尽管它们使用起来，体验感都很相似，都是使用一堆类名，但是 Tailwind CSS 比 Bootstrap5 安装起来要麻烦些，不过 Tailwind CSS 更加现代好看。其实我也挺喜欢 Bootstrap5 的设计的，具体看个人，本文使用 **Tailwind CSS**。

插一嘴更新 - v1.4(增量版) - 有关CSS最新内容(2025.08.19)

现代网页开发 - 前端与CSS框架的关系。如何快速开发一个好看实用的页面，**CSS框架(组件库)**的正确用法。

CSS框架 事实上就是给你快速构建网站用的，也包括功能。比如说CSS框架提供了一个导航栏，你只需要在他们官网把代码复制下来，之后更改关键点就好了。比如说下面这个框架，就提供了很多功能

TailwindCSS 和 Bootstrap5(引入) 比较频繁使用类名，但是如果希望快速开发网页的话，并且代码不混乱(很多类名)，CSS框架的精髓就应该是标签。如下面的css，如果我们需要卡片样式，就引入并使用 `<Card>` 标签，他们提供一些属性来DIY。这一点 **AntDesign** 做得很好。

AntDesign - <https://ant.design/>

ui.shadcn - <https://ui.shadcn.com/docs/>

例如，我们想要构建一个 图片轮询/个人信息 页面，只需要把 CSS框架 里面提供的组件复制到本地，之后写 `fetch` 从后端取得数据，填入 CSS框架的“萝卜坑”里就行了。

现代网页开发就是那么简单。

派别之争

但这其实是一个派别之争，我在 现代网页开发 教程里面书写的习惯是 - **类名 - 功能优先(Utility-First)** 而本文的习惯是 **标签 - 组件优先(Component-First)**。

如果你希望你的代码不要出现太多类名导致混乱，并且更加快速地构建一个完美，好看的页面，那就可以试试 **标签 - 组件优先(Component-First)**。

你也可以亲切将这种 **CSS框架** 称之为 **CSS组件库**。

Tailwind CSS：核心思想是Utility-First（功能优先）。它的精髓在于类名。你不需要 `Card` 标签，只需要使用一系列原子化的类名（如 `flex`, `bg-white`, `rounded-lg`）来直接构建卡片。你无法直接使用 `<Card>` 标签，你需要自己组合类名来创建卡片样式。

Bootstrap 5: 介于两者之间, 但更偏向 组件优先。它提供了像 `.card`、`.btn` 这样的组件类, 你只需要在你的 `<div>` 或 `<button>` 上添加这些类, 就可以得到一个预设样式的组件。它也有一些功能类 (如 `d-flex`), 但其核心仍然是组件类。

Ant Design: 核心思想是 Component-First (组件优先)。它的精髓在于标签。它提供了像 `<Card>`、`<Button>` 这样的 React 组件。你不需要关心背后的 CSS 类名, 只需要引入这些组件, 然后通过它们的属性 (props) 来定制样式和行为, 例如 `<Card title="My Card">`。

开发流程

1. **找到合适的模块/组件:** 从 UI 框架或组件库中, 找到你需要的模块或组件 (如导航栏、表单、表格、按钮)。
2. **复制粘贴:** 将这些组件的代码粘贴到你的项目中。这些组件通常已经包含了样式和基础功能。
3. **获得数据:** 在组件内部, 使用 `fetch` 或其他 HTTP 客户端库 (如 `Axios`) 向后端发起请求, 获取数据。
4. **填充数据:** 将从后端获取的数据, 通过 `props` 或状态管理, 动态地填充到组件的相应部分。

你会得到什么?

这个流程让你能够专注于业务逻辑和数据流, 而不用花费大量时间在 CSS 样式和基础组件的构建上。你可以把更多精力放在如何获取和展示数据, 而不是如何让按钮居中或者如何设计一个美观的功能列表。

这正是 **CSS 框架(组件库)**受欢迎的原因

接着上文 **CSS 框架(TailwindCSS / Bootstrap5) (v1.3)**

安装 Tailwind CSS

```
npm install tailwindcss @tailwindcss/vite
```

配置 Tailwind

编辑 `vite.config.js` 文件, 指定 Tailwind 的内容扫描范围:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react-swc'
import tailwindcss from '@tailwindcss/vite' // 增加这一行
```

```
export default defineConfig({
  plugins: [
    react(),
    tailwindcss()    // 增加这一行
  ],
})
```

添加 Tailwind 样式

在 `src/index.css` 中导入 Tailwind 的基础样式即可 (头部插入):

```
@import "tailwindcss";
```

即可应用 TailwindCSS

安装 Bootstrap 5

```
npm install bootstrap
```

导入 Bootstrap 样式

在 `src/main.jsx` 中导入 Bootstrap 的 CSS 文件:

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

使用 Bootstrap 样式

编辑 `src/App.jsx`, 使用 Bootstrap 样式:

```
function App() {
  return (
    <div className="container mt-5">
      <h1 className="text-primary">Welcome to Bootstrap 5 in Vite!</h1>
      <button className="btn btn-primary">Click Me</button>
    </div>
  );
}

export default App;
```

组件 - 现代网页开发 重要思维之一

在开始路由前，我们需要先讲组件。虽然网站是像剥洋葱一样剥开的，但是我们学的话，剥洋葱地学很容易乱，所以要从洋葱里面开始(有点像，把洋葱当成积木，拼起来)。

什么是组件？

在一个页面上面，有很多框框，比如说左边是你的头像和个人信息，中边是新闻，最右边是搜索。

类似于...Twitter, Facebook 之类的。

现代开发，组件的应用是必不可少的，重要的，先进的！

导航栏和公告栏 - 组件

首先我们创建一个目录用于存放组件， `src/components`，组件还有个优点就是，复用性。

如果曾经有开发过网站的经验，应该都会就 每个页面都有的「导航栏」，「回到顶端」等功能，进行剥离，放到某个文件夹(如组件文件夹)，之后被各大页面复用(引用)。就无须10个页面，就写(复制粘贴)10次的导航栏。

组件也就是这样的思维，不过因为 CSS 框架和前端框架的加持，会使得组件用起来，更加现代，优雅。

我们在 `src/components` 下创建两个文件，分别为 `src/components/AnnouncementBar.jsx` (公告行)，`src/components/Navbar.jsx` (导航栏)

公告栏 `AnnouncementBar.jsx`

```
import React, { useState, useEffect } from "react";

const AnnouncementBar = () => {
  const [announcement, setAnnouncement] = useState("");

  // 获取公告内容
  useEffect(() => {
    const fetchAnnouncement = async () => {
      try {
        const response = await fetch("http://localhost:3000/data"); // 后端
```

API 地址

```
const data = await response.json();
setAnnouncement(data.readData.announcement); // 剥离数据

} catch (error) {
  console.error("Failed to fetch announcement:", error);
  setAnnouncement("无法加载公告, 请稍后重试。"); // 错误时
}

};
fetchAnnouncement();
}, []);

// 如果没有公告, 则不显示组件
if (announcement == null) {
  return null;
}

return (
  <div className="bg-blue-500 text-white text-center py-2 px-4">
    <p className="text-sm font-medium">{announcement}</p>
  </div>
);
};

export default AnnouncementBar;
```

导航栏 Navvar.jsx

代码用到一个 LOGO, logo 位于 `/public/logo.png`, 随便找一个替代即可。

```
import React, { useState } from "react";

// 子组件: 箭头图标
const ArrowRightIcon = () => (
  <svg
    xmlns="http://www.w3.org/2000/svg"
    viewBox="0 0 20 20"
    fill="currentColor"
    className="w-5 h-5 ml-1"
  >
    <path
```

```

        fillRule="evenodd"
        d="M3 10a.75.75 0 01.75-.75h10.638L10.23 5.29a.75.75 0 111.04-1.08l5.5
5.25a.75.75 0 010 1.08l-5.5 5.25a.75.75 0 11-1.04-1.08l4.158-
3.96H3.75A.75.75 0 013 10z"
        clipRule="evenodd"
    />
</svg>
);

```

// 子组件：菜单图标

```

const MenuIcon = () => (
  <svg
    className="block h-6 w-6"
    xmlns="http://www.w3.org/2000/svg"
    fill="none"
    viewBox="0 0 24 24"
    stroke="currentColor"
    aria-hidden="true"
  >
    <path
      strokeLinecap="round"
      strokeLinejoin="round"
      strokeWidth="2"
      d="M4 6h16M4 12h16M4 18h16"
    />
  </svg>
);

```

// 子组件：关闭图标

```

const CloseIcon = () => (
  <svg
    className="block h-6 w-6"
    xmlns="http://www.w3.org/2000/svg"
    fill="none"
    viewBox="0 0 24 24"
    stroke="currentColor"
    aria-hidden="true"
  >
    <path
      strokeLinecap="round"
      strokeLinejoin="round"
      strokeWidth="2"
      d="M6 18L18 6M6 6l12 12"
    />
  </svg>
);

```



```

function Navbar() {
  const [isMobileMenuOpen, setIsMobileMenuOpen] = useState(false);

  // 导航链接
  const navLinks = [
    { name: "主页", href: "#" },
    { name: "关于我们", href: "#" },
    { name: "测试页面", href: "#" },
  ];

  // 公共样式
  const linkStyle =
    "text-gray-700 hover:text-indigo-600 px-3 py-2 rounded-md text-sm font-medium";

  // 渲染导航链接
  const renderNavLinks = () =>
    navLinks.map((link) => (
      <a key={link.name} href={link.href} className={linkStyle}>
        {link.name}
      </a>
    ));

  return (
    <nav className="bg-white shadow-sm">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex items-center justify-between h-16">
          {/* 左侧导航链接 */}
          <div className="hidden md:flex space-x-4">{renderNavLinks()}</div>

          {/* 中间 Logo */}
          <div className="flex-shrink-0">
            <a href="/" aria-label="Home">
              
            </a>
          </div>

          {/* 右侧登录链接 */}
          <div className="hidden md:flex items-center">
            <a href="#" className={` ${linkStyle} flex items-center`} >
              Log in
            </a>
          </div>
        </div>
      </div>
    </nav>
  );
}

```

```

        <ArrowRightIcon />
      </a>
    </div>

    { /* 移动端菜单按钮 */ }
    <div className="md:hidden">
      <button
        onClick={() => setIsMobileMenuOpen(!isMobileMenuOpen)}
        className="bg-white p-2 rounded-md text-gray-400 hover:text-gray-500 hover:bg-gray-100 focus:outline-none focus:ring-2 focus:ring-indigo-500"
        aria-expanded={isMobileMenuOpen}
      >
        <span className="sr-only">Toggle menu</span>
        {isMobileMenuOpen ? <CloseIcon /> : <MenuIcon />}
      </button>
    </div>
  </div>
</div>

{ /* 移动端菜单 */ }
{isMobileMenuOpen && (
  <div className="md:hidden">
    <div className="px-2 pt-2 pb-3 space-y-1">{renderNavLinks()}</div>
    <div className="border-t border-gray-200 pt-4 pb-3">
      <a href="#" className={` ${linkStyle} flex items-center`} >
        Log in
        <ArrowRightIcon />
      </a>
    </div>
  </div>
)}
</nav>
);
}

export default Navbar;

```

挂载

完成上面的一切，现在要做的就是挂载了。

但是我们需要挂载到路由上，但是路由页面还没有写，所以先放一放，把路由解决，之后挂载到路由上，再将路由挂载到 App.jsx (主应用) 上。

层级关系 | 「组件」 ==挂载==> 「路由页面」 ==挂载==> 「主页」(主应用)

React-Router (路由)使用

前后端路由 - 概念：

前后端路由是不同的，**后端路由** 指的是 API 接口，一般一个路由专注于做一件事，比如说注册路由，index路由，文章路由等等。

前端路由 是为了在浏览器中实现无刷新，即时切换，无需每次点击一个页面都要更新一次页面。

比如说，我们有一个页面，页面下面有三个分页，首页，关于我们，测试页面。那么在该页面下面点击分页，就在指定的区域加载分页出来，而不是“跳转”，“重载”整个页面。

像后端一样接收参数(params)! (v1.5) 2025.08.29

前端路由完全可以像是后端一样接收params，也就是 `/:key` 这种路由，后端通过 `const key = req.params.key;` 的方式获得变量。其实在前端也支持这样的操作，在接收到这样的 params 变量后，就可以设计代码逻辑去 Fetch 不同的后端路由，或者是渲染不同的内容。

如何使用？

为了让你有比较清晰的认知，请继续往下看。你需要先知道 App.jsx 挂载路由，和路由设计才能继续这一步。如果你已经会了，就请直接移步到文末。如果你还不会，请不要直接看，因为可能会看懵，这对学习编程时候需要的实践自信和理解自信是毁灭的打击，会导致你学不好编程！（应该这样吧）

1. 创建页面组件

在 `src/pages` 文件夹中创建页面组件。

Home.jsx (首页)

```
import React from "react";
import AnnouncementBar from '../components/AnnouncementBar';
import Navbar from '../components/Navbar';
```

```

const Home = () => {
  return (
    <>
      <div>
        <AnnouncementBar />
        <Navbar />

        <div className="h-screen bg-gradient-to-r from-blue-500 to-purple-500 flex items-center justify-center">
          <div className="text-center text-white">
            <h1 className="text-5xl font-bold mb-6">
              Welcome to Our Application
            </h1>
            <p className="text-lg text-gray-200 max-w-xl mx-auto">
              This is a modern React application powered by
              Tailwind CSS. Explore the features and enjoy a seamless user experience.
            </p>
          </div>
        </div>
      </div>
    </>
  );
};

export default Home;

```

About.jsx (关于页)

```

import React from "react";
import AnnouncementBar from "../components/AnnouncementBar";
import Navbar from "../components/Navbar";

const About = () => {
  return (
    <>
      <div>
        <AnnouncementBar />
        <Navbar />

        <div className="min-h-screen flex flex-col items-center justify-center bg-white text-gray-800">
          <h1 className="text-4xl font-bold mb-4">About Us</h1>
          <p className="text-lg text-gray-600 text-center max-w-2xl">
            Welcome to our application! We are a team dedicated

```

to delivering high-quality solutions that empower our users to achieve their goals. Our mission is to blend simplicity, beauty, and functionality in all that we do.

```
        </p>
        <div className="mt-6">
          <a
            href="https://example.com"
            target="_blank"
            rel="noopener noreferrer"
            className="px-6 py-2 bg-blue-500 text-white rounded-
md hover:bg-blue-600 transition">
            Learn More
          </a>
        </div>
      </div>
    </div>
  </>
);
};

export default About;
```

NotFound.jsx (404 页面)

```
import React from "react";
import AnnouncementBar from '../components/AnnouncementBar';
import Navbar from '../components/Navbar';

import { Link } from "react-router-dom"; // 用来返回上一页（用a标签会刷新页面）

const NotFound = () => {
  return (
    <>
      <div>
        <AnnouncementBar />
        <Navbar />

        <div className="h-screen flex flex-col items-center justify-
center bg-gray-100 text-gray-800">
          <h1 className="text-9xl font-bold text-blue-500">404</h1>
          <h2 className="text-2xl font-semibold mt-4">Oops! Page not
found.</h2>
          <p className="text-gray-500 mt-2">
            Sorry, the page you are looking for doesn't exist or has
            been moved.
          </p>
        </div>
      </div>
    </>
  );
};
```

```

        </p>
        <Link
          to="/"
          className="mt-6 px-6 py-2 bg-blue-500 text-white
rounded-md hover:bg-blue-600 transition">
          Back to Home
        </Link>
      </div>
    </div>
  </div>
);
};

export default NotFound;

```

2. 定义路由结构

在 `src/App.jsx` 中挂载路由。

App.jsx

```

import React from 'react';
// 请注意，这条引用用了重命名，把 BrowserRouter 重命名成了 Router //
import { BrowserRouter as Router, Routes, Route } from "react-router-dom";

import Home from './pages/Home';
import About from './pages/About';
import NotFound from './pages/NotFound'

import './App.css';

const App = () => {
  return (
    <>
      <Router>
        <div>
          <Routes>
            <Route path="/" element={<Home />} />
            <Route path="/About" element={<About />} />
            <Route path="/*" element={<NotFound />} />
          </Routes>
        </div>
      </Router>
    </>
  );
};

```

```
    </>
  );
};

export default App;
```

上面为什么用 * 号？认真把下面看完。

路由标签的区别。BrowserRouter、Routes和Route

上面的 `BrowserRouter` 被重命名为了 `Router`，代码实际的情况是：

```
...
...
<BrowserRouter>
  <div>
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/About" element={<About />} />
      <Route path="/*" element={<NotFound />} />
    </Routes>
  </div>
</BrowserRouter>
...
...
```

BrowserRouter 提供上下文

- 管理路由的核心逻辑，监听浏览器地址栏的变化（如用户切换页面、点击返回按钮）。
- 将当前 URL 状态传递给子组件（`Routes` 和 `Route`）。

Routes 定义规则集

- 收集所有的 `Route` 规则，检查当前路径是否匹配某条规则。
- 如果找到匹配的规则，就渲染对应的组件。

Route 定义具体规则

- `path`：表示匹配的路径。
- `element`：表示路径对应的组件。
- 例如：
 - `/` 匹配 `Home` 组件。
 - `/about` 匹配 `About` 组件。

- * 匹配所有未定义的路径（通常用作 404 页面）。

为什么用 * 号？

* 号匹配了所有的路径地址，但是因为我们没有定义这些路径，所以其他路径自然而然就返回不出来模版，那么就把这些被匹配到的所有、找不到模版的路径，统一返回 404(NotFound) 模版(路由)页面

从 URL 到页面渲染的过程：

假设用户访问了 /About :

1. BrowserRouter 检测 URL:

- 当前路径是 /About 。
- 将 URL 状态传递给 Routes 。

2. Routes 匹配路径:

- 检查所有的 Route :
 - / 不匹配。
 - /About 匹配 - 返回 element属性 选择的模版。
 - * (404) 不匹配 (因为上面找到有匹配的)。
- 找到 /About 的匹配规则。

3. Route 渲染组件:

- 根据 /About 的规则，渲染 <About /> 组件。

最终，浏览器显示 About 页面内容。

假设用户访问了 /Axxt 不存在的路由(路径):

1. BrowserRouter 检测 URL:

- 当前路径是 /Axxt 。
- 将 URL 状态传递给 Routes 。

2. Routes 匹配路径:

- 检查所有的 Route :
 - / 不匹配。
 - /About 不匹配。
 - * (404) 匹配 - 都找不到匹配的吧，来吧，我来匹配你 。
- 找不到 /Axxt 的匹配规则，那就匹配到: * 。

3. Route 渲染组件:

- 根据 * 的规则，渲染 <NotFound /> 组件。

最终，浏览器显示 NotFound 页面内容。

3. 更改 导航组件 - 重要

我们需要更改导航组件 `src/components/Navbar.jsx` 的跳转地址和方式，目前跳转的地址是 `#`，并且使用 `<a href='#' >` (a标签) 跳转，这是不可取的，因为 `a` 标签跳转，是会刷新页面的，这并不符合我们对于页面不刷新切换路由的需要

```
import React, { useState } from "react";
import { Link } from "react-router-dom"; // 导入 Link 组件，用于代替会刷新页面的
a 标签

// 子组件：箭头图标
const ArrowRightIcon = () => (
  <svg
    xmlns="http://www.w3.org/2000/svg"
    viewBox="0 0 20 20"
    fill="currentColor"
    className="w-5 h-5 ml-1"
  >
    <path
      fillRule="evenodd"
      d="M3 10a.75.75 0 01.75-.75h10.638L10.23 5.29a.75.75 0 11.04-1.08l5.5
5.25a.75.75 0 010 1.08l-5.5 5.25a.75.75 0 11-1.04-1.08l4.158-
3.96H3.75A.75.75 0 013 10z"
      clipRule="evenodd"
    />
  </svg>
);

// 子组件：菜单图标
const MenuIcon = () => (
  <svg
    className="block h-6 w-6"
    xmlns="http://www.w3.org/2000/svg"
    fill="none"
    viewBox="0 0 24 24"
    stroke="currentColor"
    aria-hidden="true"
  >
    <path
      strokeLinecap="round"
      strokeLinejoin="round"
      strokeWidth="2"
      d="M4 6h16M4 12h16M4 18h16"
    />
  </svg>
);
```

```

    </svg>
  );

  // 子组件：关闭图标
  const CloseIcon = () => (
    <svg
      className="block h-6 w-6"
      xmlns="http://www.w3.org/2000/svg"
      fill="none"
      viewBox="0 0 24 24"
      stroke="currentColor"
      aria-hidden="true"
    >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"
        strokeWidth="2"
        d="M6 18L18 6M6 6L12 12"
      />
    </svg>
  );

  function Navbar() {
    const [isMobileMenuOpen, setIsMobileMenuOpen] = useState(false);

    // 导航链接 ////////// 需要更改 //////////
    const navLinks = [
      { name: "主页", path: "/" }, // 更改为前端路由页面的URL地址
      { name: "关于我们", path: "/About" }, // 与 App.js 相关, path 指向
      { name: "测试页面", path: "/NotFound" },
      // 这里 NotFound 路径其实不存在, 因为 App.js 里面定义的是 * 号, 不过 * 匹配了所有
      // 地址, 之后就返回了 404(NotFound)模版(路由)页面。
      // 所以这里爱啥啥, 但是因为是测试页面, 所以直接采用 /NotFound 路径来保持美观罢了。
    ];

    // 公共样式
    const linkStyle =
      "text-gray-700 hover:text-indigo-600 px-3 py-2 rounded-md text-sm font-medium";

    // 渲染导航链接 ////////// 需要更改 //////////
    const renderNavLinks = () =>
      navLinks.map((link) => ( // 为了避免 原生a标签 的跳转(刷新)网页问题
        <Link key={link.name} to={link.path} className={linkStyle}>
          {link.name}
        </Link> // 这里的 a 标签换成 React-Route 提供的 Link 标签
      ));
  }

```

```

));

return (
  <nav className="bg-white shadow-sm">
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
      <div className="flex items-center justify-between h-16">
        {/* 左侧导航链接 */}
        <div className="hidden md:flex space-x-4">{renderNavLinks()}</div>

        {/* 中间 Logo */}
        <div className="flex-shrink-0">
          <Link to="/" aria-label="Home">
            
          </Link>
        </div>

        {/* 右侧登录链接 */}
        <div className="hidden md:flex items-center">
          <Link to="#" className={` ${linkStyle} flex items-center`} >
            Log in
            <ArrowRightIcon />
          </Link>
        </div>

        {/* 移动端菜单按钮 */}
        <div className="md:hidden">
          <button
            onClick={() => setIsMobileMenuOpen(!isMobileMenuOpen)}
            className="bg-white p-2 rounded-md text-gray-400 hover:text-gray-500 hover:bg-gray-100 focus:outline-none focus:ring-2 focus:ring-indigo-500"
            aria-expanded={isMobileMenuOpen}
          >
            <span className="sr-only">Toggle menu</span>
            {isMobileMenuOpen ? <CloseIcon /> : <MenuIcon />}
          </button>
        </div>
      </div>
    </div>

    {/* 移动端菜单 */}
    {isMobileMenuOpen && (

```

```

    <div className="md:hidden">
      <div className="px-2 pt-2 pb-3 space-y-1">{renderNavLinks()}</div>
      <div className="border-t border-gray-200 pt-4 pb-3">
        <Link to="#" className={` ${linkStyle} flex items-center`} >
          Log in
          <ArrowRightIcon />
        </Link>
      </div>
    </div>
  )}
</nav>
);
}

export default Navbar;

```

4. 挂载 React 应用

在 `src/main.jsx` 中使用 `ReactDOM.createRoot` 将 React 应用挂载到真实的 DOM 节点。

main.jsx

```

import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

// 下面代码同样可行，不完整导入 React 和 ReactDOM，只导入需要用的 React.StrictMode
// 和 ReactDOM.createRoot
// import { StrictMode } from 'react'
// import { createRoot } from 'react-dom/client'
// import App from './App.jsx'
// import './index.css'

// createRoot(document.getElementById('root')).render(
//   <StrictMode>

```

```
//      <App />
//    </StrictMode>,
// );
```

总结说明一下

React Router 的路由挂载 - 路由标签

1. BrowserRouter :

- 是 React Router 的路由上下文组件，必须包裹在应用的最外层，负责管理路由的状态和历史记录。
- 通过它，子组件可以使用路由功能。

2. Routes :

- 用于定义路由规则，包含多个 Route 。

3. Route :

- 定义每个路由的路径和对应的组件。
- path 属性表示 URL 路径，element 属性表示要渲染的组件。

4. Link :

- 用于创建导航链接，类似于 HTML 的 <a> 标签，但不会刷新页面，而是通过 React Router 实现页面切换。

ReactDOM 的作用 - 挂载标签

ReactDOM 是 React 提供的一个库，用于将 React 组件渲染到真实的 DOM 节点中。

1. ReactDOM.createRoot :

- 是 React 18 提供的新 API，用于创建 React 应用的根节点。
- 替代了 React 17 中的 ReactDOM.render 方法，支持并发模式并提升性能。

2. 挂载流程:

- **ReactDOM.createRoot(document.getElementById("root")) :**
 - 找到 HTML 页面中 id="root" 的 DOM 节点，作为 React 应用的挂载点。
- **.render(<App />) :**
 - 将 React 的根组件（如 App）渲染到 root 节点中，启动整个应用。

3. 作用:

- 将 React 的虚拟 DOM 转换为真实 DOM，显示在浏览器中。
 - 实现 React 状态与真实 DOM 的同步更新，确保用户界面动态响应数据变化。
-

登陆路由

理解完成上面的东西，基本上也就已经入门前端开发了(吧?)

所以下面就过一遍，写一个登陆路由和注册来对接后端，之后把修改一下 `Home.jsx`，列出一下最近10篇文章。

⚠ 警告：对于 后端路由 `admin.js`，值得注意的是，管理员和普通用户在一个表，且没有区分的列。意味着普通用户 = 管理员

所以我们要对数据库做出一些更改：下面有两个方案

方案 1：管理员和普通用户放在同一个表

在用户表中添加一个 角色字段 (如 `role` 或 `is_admin`)，用来区分用户的身份。

表结构示例

users

id	username	email	role
1	Alice	alice@test.com	user
2	Bob	bob@test.com	admin
3	Carol	carol@test.com	user

- **role 字段：**

- 可以存储用户的角色 (如 `user`、`admin` 等)。
- 如果只是简单区分管理员和普通用户，可以用布尔值字段 (如 `is_admin`) 代替。

is_admin

id	username	email	is_admin
1	Alice	alice@test.com	false
2	Bob	bob@test.com	true

优点

1. 简单易维护：

- 所有用户信息都在一个表中，便于查询和管理。

2. 便于扩展：

- 如果需要在支持更多角色（如 `editor`, `moderator` 等），只需扩展 `role` 字段即可。

3. 性能优化：

- 避免多表查询或关联操作，查询性能较高。

缺点

1. 字段语义复杂：

- 如果用户的角色逻辑复杂，可能会导致 `role` 字段值过于多样，增加维护难度。

2. 权限控制逻辑复杂：

- 需要在代码中根据 `role` 或 `is_admin` 动态判断权限，权限管理可能比较分散。

方案 2：管理员和普通用户分两个表

将管理员和普通用户分开存储在不同的表中，例如 `users` 和 `admins`。

表结构示例

users

id	username	email
1	Alice	alice@test.com
2	Carol	carol@test.com

admins

id	username	email
1	Bob	bob@test.com

优点

1. 数据结构清晰：

- 管理员和普通用户分开存储，逻辑更直观，权限逻辑更容易管理。

2. 便于权限扩展：

- 如果管理员的数据结构与普通用户差异较大，可以为管理员单独设计专属字段，而不影响普通用户的结构。

缺点

1. 查询复杂度增加:

- 查询用户数据时需要判断是在 `users` 表还是 `admins` 表, 增加开发复杂度。

2. 冗余字段:

- 两个表可能会有大量重复字段 (如 `username`、`email` 等), 导致数据冗余。

3. 扩展不灵活:

- 如果支持更多的角色 (如 `editor`, `moderator`), 可能需要创建更多的表, 数据设计变得不够灵活。

正文 - 认证 你是你(管理员)

如果你希望有更多的鉴权应用场景, 那么多用户身份无疑是最好的选择。但是我的网站只有用户和管理员之分, 所以我用的是「真或假」。

我们取用 **方案一**, 我们需要在 `Users` 这个数据库里面加入一个列(role), 直接采用 **布尔值** 的方式区分管理员和普通用户(毕竟只是小项目), 但是我建议直接通过 字段(列 - role) 的值来判断是否是管理员或者其他用户, 这样的数据库和代码, 对于后期权限的扩展, 更加灵活。

```
ALTER TABLE users ADD COLUMN role BOOLEAN NOT NULL DEFAULT FALSE;
```

之后, 把管理员的 `role` 字段, 改成 `1(True)` 即可。

我们需要在 `/models/User.js` 模型中增加这个列

`/models/User.js`

```
const { DataTypes } = require('sequelize');
const sequelize = require('../database/database');

const User = sequelize.define('User', {
  id: {
    type: DataTypes.INTEGER,
    autoIncrement: true,
    primaryKey: true,
  },
  username: {
    type: DataTypes.STRING,
    allowNull: false,
    unique: true,
  },
},
```



```

password: {
  type: DataTypes.STRING,
  allowNull: false,
},
email: {
  type: DataTypes.STRING,
  allowNull: false,
  unique: true,
},
is_member: {
  type: DataTypes.BOOLEAN,
  defaultValue: true,
},
role: { // 追加刚刚增加的列
  type: DataTypes.BOOLEAN,
  defaultValue: false,
}
}, {
  timestamps: true,
});

module.exports = User;

```

/routes/login.js 这个登陆路由是普通用户的...但是，我们也可以用。

~~但是，/admin 需要改变了，可不能让那群低等的用户，进入我们高雅的上层大厅~~

不好意思，刚刚被顶号了。好了，其实现代的网页程序，一般都不在后台再整一个登陆页面出来了，而是直接用户和管理员走一个登陆通道，但不同的是，普通用户访问「私人路由」，如「后台路由」这样的路由，会直接返回 404-NotFound 的页面，或者直接告诉你权限不足(前者明显安全性更高)。

上面是笼统的说法，如果你希望深入了解为什么，可以往下看，否则跳过。

1. 用户和管理员通常共用一个登录通道

在现代的网页程序中，用户和管理员通常通过 **同一个登录界面** 进行认证。以下是这样设计的原因：

为什么不区分登录页面？

1. 简化开发：

- 使用统一的接口和登录逻辑可以减少重复开发工作，不需要为管理员单独设计一个登录页面或接口。
- 用户和管理员的认证流程本质上是一样的（验证用户名/密码），区别在于登录后的权限控制。

2. 安全性:

- 如果管理员有独立的登录页面，会暴露出管理员专属的入口，可能会吸引攻击者进行暴力破解或其他攻击。
- 共用一个登录入口，可以隐藏管理员角色，减少攻击的可能性。

3. 用户体验:

- 对于拥有多种角色的用户（如既是普通用户又是管理员），共用一个登录界面可以避免混淆。

如何区分用户和管理员?

- 登录成功后，后端会根据用户的 **角色信息**（如 `role` 字段或权限列表）返回对应的权限。
- 前端根据返回的权限信息动态展示不同的页面或功能。例如：
 - 普通用户可以访问「用户中心」。
 - 管理员可以访问「后台管理」。

2. 权限管理：私人路由与后台路由的处理方式

权限管理是现代网页程序中的核心部分，重点是如何处理 **普通用户访问管理员路由** 的情况。

两种常见处理方式

1. 返回 404 页面（更安全的做法）:

- 如果普通用户尝试访问管理员的专属路由（如 `/admin/dashboard`），直接返回 404 页面。
- 这样可以隐藏管理员路由的存在，避免暴露后台系统的入口。
- 优点：
 - 提高安全性，攻击者无法轻易探测出哪些路由是管理员路由。
- 实现方式：
 - 后端根据用户权限控制是否返回路由内容。
 - 前端通过路由守卫（如 React Router 的 `PrivateRoute`）拦截未授权用户。

示例（前端实现）:

```
const PrivateRoute = ({ component: Component, ...rest }) => {
  const userRole = getUserRole(); // 从全局状态或后端获取用户角色
  return (
    <Route
      { ...rest }
      render={() =>
        userRole === "admin" ? (
          <Component { ... props } />
        ) : (
          <Redirect to="/404" /> // 未授权用户重定向到 404 页面
        )
      }
    />
  )
}
```

```
    />  
  );  
};
```

2. 返回权限不足提示：

- 如果普通用户访问了管理员路由，显示一个提示页面（如「权限不足」或「无权访问」）。
- 优点：
 - 提供明确的信息反馈，用户知道自己没有权限访问该页面。
- 缺点：
 - 通过返回权限不足的页面，攻击者可以推测出后台路由的存在，从而增加暴力破解的风险。
- 实现方式：
 - 后端在返回路由数据时，校验用户权限，如果无权限，返回 403 错误（Forbidden）。
哪种方式更好？

- 返回 404 页面：适用于对安全性要求较高的系统（如企业管理系统、金融系统）。
- 返回权限不足提示：适用于对用户体验要求更高的系统（如电商平台、内容管理系统）。

3. 前端与后端的权限分工

后端权限控制

- 后端是权限控制的核心，因为前端的代码可能会被攻击者反编译和修改。
- 后端需要对每个请求进行严格的权限校验：
 - 验证用户的登录状态和角色。
 - 如果用户无权访问某个资源，直接返回 403（权限不足）或 404（资源不存在）。

前端权限控制

- 前端的权限控制主要是提升用户体验，避免不必要的页面加载和请求。
- 常见的前端权限控制方法：
 - 路由守卫：
 - 在用户访问某些路由前，校验其权限，如果无权限则重定向到 404 页面或显示权限不足提示。
 - 动态菜单渲染：
 - 根据用户权限动态渲染页面菜单和功能按钮，普通用户看不到管理员特有的功能。

4. 补充：如何提升安全性？

1. 隐藏敏感路由：

- 不要在前端代码中暴露管理员路由（如 /admin/dashboard）。
- 使用后端返回的动态路由表，让前端根据权限动态渲染路由。

2. 使用中间件校验权限：

- 在后端通过中间件（如 Express 中的 middleware）统一校验请求权限，避免每个路由都重复实现权限逻辑。

3. 限制敏感信息的暴露：

- 后端接口只返回当前用户有权限访问的数据。
- 对管理员的接口请求进行严格的身份验证（如 IP 白名单、二次认证等）。

5. 总结

现代网页程序中通常设计为：

1. 用户和管理员共用一个登录通道，登录后通过权限字段区分身份。
2. 对普通用户访问管理员专属路由的处理：
 - 安全性优先：返回 404 页面，隐藏路由存在。
 - 用户体验优先：显示权限不足提示。推荐：

- 如果安全性是首要考虑（如管理系统），优先选择返回 404 页面。
- 如果用户体验更重要（如电商、内容管理系统），可以返回权限不足提示。

通过后端权限校验 + 前端路由守卫的组合，既能保证安全性，又能提升用户体验。

方案一

首先，我们更改一下 routes/login.js 下发的 Token，因为它只携带了，user.id 和 user.username，我们需要多携带一个 role，之后「私人路由」解析Token，就可以获得一个 role 值，之后对这个值进行判断就行。

```
const express = require('express');
const bcrypt = require('bcrypt');
const { User } = require('../models');
const { generateToken } = require('../utils/jwt');

const router = express.Router();

router.post('/', async (req, res) => {
  const { username, password } = req.body;

  try {
    // 查找用户
    const user = await User.findOne({ where: { username } });

    if (!user) {
```

```

    return res.status(404).json({ error: 'User not found' });
  }

  // 验证密码
  const isPasswordValid = await bcrypt.compare(password, user.password);
  if (!isPasswordValid) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }

  // 生成 JWT /////////////// 追加 user.role ///////////////
  const token = generateToken({ id: user.id, username: user.username,
    role: user.role });

  res.json({ token });
} catch (error) {
  console.error(error);
  res.status(500).json({ error: 'Something went wrong' });
}
});

module.exports = router;

```

之后我解释一下 routes/admin.js

```

const express = require('express');
const { authenticateJWT } = require('../middlewares/auth');

const router = express.Router();

// authenticateJWT 这个中间件解密了 Token, 并通过了 req.user = decoded 这种方式赋值
// 我们通过 req.user.role 的方式去解析出 role 字段
router.get('/', authenticateJWT, (req, res) => {
  console.log(req.user.role); // 可以打印出 role 情况
  res.json({ message: 'Welcome to the admin panel', user: req.user });
});

module.exports = router;

```

带 Token(管理员) 访问后, 返回的数据 就是这样的:

```

{
  "message": "Welcome to the admin panel",
  "user": {
    "id": 1,

```

```
    "username": "admin",
    "role": true,
    "iat": 174xxxx359,
    "exp": 174xxxx759
  }
}
```

带普通用户Token 访问后, 返回的数据 就是这样的:

```
{
  "error": "Pages Not Found."
}
```

下面是 routes/admin.js 布尔值版本(因为我的数据库设计是布尔值的):

```
const express = require('express');
const { authenticateJWT } = require('../middlewares/auth');

const router = express.Router();

// authenticateJWT 这个中间件解密了 Token
router.get('/', authenticateJWT, (req, res) => {
  // 对什么都不携带的嗅探返回 404
  if (!req.user || !req.user.role) {
    return res.status(404).json({ error: 'Pages Not Found.' });
  };

  // 对管理员返回 正常页面
  if (req.user.role == true) {
    res.json({ message: 'Welcome to the admin panel', user: req.user });
  };

  // 对普通用户返回 404
  if (req.user.role == false) {
    return res.status(404).json({ error: 'Pages Not Found.' });
  };
});

module.exports = router;
```

如果你希望有更多的鉴权应用场景, 那多用户身份无非是最好的选择。但是我的网站只会有用户和管理员之分, 所以我用的是「真或假」。

方案二

```
const express = require('express');
const { authenticateJWT } = require('../middlewares/auth');

const router = express.Router();

// authenticateJWT 中间件解密 Token
router.get('/', authenticateJWT, (req, res) => {
  // 检查 req.user 和 req.user.role 是否存在
  if (!req.user || !req.user.role) {
    return res.status(403).json({ error: 'Unauthorized: Invalid token or role missing' });
  }

  // 判断角色
  if (req.user.role === 'admin') {
    // 管理员角色
    return res.json({ message: 'Welcome to the admin panel', user: req.user });
  } else if (req.user.role === 'user') {
    // 普通用户角色
    return res.status(404).json({ error: 'Pages Not Found.' });
  }

  // 其他未知角色
  return res.status(403).json({ error: 'Forbidden: Role not recognized' });
});

module.exports = router;
```

React 登陆页面

我们需要写一个登陆页面，使用测试工具测试 Token 都要自己手动带上，我们写一个前端的登陆页面，需要完成 Token 的自携带才行。

自动携带 Token 可以通过「**Axios**」请求拦截器自动将 Token 添加到每次请求的 Authorization 请求头中。但是我选择用原生的 Fetch，但是使用它，我们需要自己封装一个功能出来。

设置 Token 到 Cookie

因为把 Token 设置到 LocalStorage 不安全，容易受到 XSS 攻击，所以为了支持 HttpOnly Cookie，我们需要在服务器设置 Cookie。

==之前的代码，直接 `res.json({ token })`；是为了方便查看到返回的 **Token**，之后做测试，而代码 **投入使用后**，直接 `res.cookie` 设置完 Token 到 Cookie 之后，返回 `status:200`，`message:'Login Successful!'` 就行了。==

routes/login.js

```
... ..
...
// 验证密码
const isPasswordValid = await bcrypt.compare(password, user.password);
if (!isPasswordValid) {
  return res.status(401).json({ error: 'Invalid credentials' });
}

// 生成 JWT
const token = generateToken({ id: user.id, username: user.username, role:
user.role });

// res.json({ token });

// 设置 HttpOnly Cookie
res.cookie("authToken", token, {
  httpOnly: true,      // 禁止 JavaScript 访问
  secure: true,         // 仅在 HTTPS 上传输
  maxAge: 14400000,     // Cookie 有效期（4 小时 - 以毫秒为单位）
  sameSite: "Strict",  // 防止 CSRF 攻击
});

res.json({ message: `Login successful! ${user.username}` })
... ..
... ..
```

上面返回的一个请求头有 `Set-Cookie`，浏览器就是靠这个设置 Cookie 的，之后会自己携带，如果失效了，就抛弃。

重新设置 CORS 策略 以及 设置 `Cookie-parser`

如果不重新设置，将会被拦截，因为本来的 CORS 设置全局太过逆天，并且没有设置 Cookie 放行。如果不设置的话，浏览器将会拦截你的请求，故登陆失败。

为什么 `req.cookie` 是未定义

- Express 默认不会解析 `cookie`，需要手动使用 `cookie-parser` 中间件。
- `cookie-parser` 会将 `req.headers.cookie` 中的原始字符串解析为一个对象，并将其挂载到 `req.cookies`。

- 安装: `npm install cookie-parser`

后端 `app.js`

```
const express = require('express');
const bodyParser = require('body-parser');
const cookieParser = require('cookie-parser'); // Cookie 解析
const indexData = require('./routes/siteintro');
const indexRoutes = require('./routes/index');
const articleRoutes = require('./routes/article');
const registerRoutes = require('./routes/register');
const authRoutes = require('./routes/login');
const adminRoutes = require('./routes/admin');

const cors = require("cors");
const app = express();

app.use(bodyParser.json());

app.use(cookieParser()); // 使用 cookie-parser 中间件

// // 全局启用 CORS
// app.use(cors());

// CORS 配置
app.use(
  cors({
    origin: "http://localhost:5173", // 前端地址
    credentials: true, // 允许携带 Cookie
  })
);

// 路由
app.use('/', indexRoutes);
app.use('/data', indexData);
app.use('/article', articleRoutes);
app.use('/register', registerRoutes);
app.use('/login', authRoutes);
app.use('/admin', adminRoutes);

const PORT = 3000;

app.listen(PORT, async () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

前端 示例代码

当访问需要身份验证的后端路由时，例如 `/api/protected`，前端仍然需要设置 `credentials: "include"`，让浏览器自动携带 `HttpOnly Cookie`。

登陆请求示例代码：

```
const login = async (username, password) => {
  try {
    const response = await fetch("http://localhost:3000/login", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({ username, password }), // 发送用户名和密码
      credentials: "include", // 确保携带和存储 HttpOnly Cookie
    });

    if (!response.ok) {
      const errorData = await response.json();
      throw new Error(errorData.error || "Login failed");
    }

    // 登录成功
    alert("Login successful!");
  } catch (error) {
    console.error("Login error:", error.message);
    alert(error.message);
  }
};
```

访问受保护的接口示例代码：

```
const fetchProtectedData = async () => {
  try {
    const response = await fetch("http://localhost:3000/api/protected", {
      method: "GET",
      credentials: "include", // 自动携带 HttpOnly Cookie (重要)
    });

    if (response.status === 401) {
      // 如果服务器返回 401, 说明 Token 已过期
      alert("Session expired. Redirecting to login...");
    }
  }
};
```

```

    window.location.href = "/login"; // 跳转到登录页面
    return;
}

if (!response.ok) {
    throw new Error("Failed to fetch protected data");
}

const data = await response.json();
console.log("Protected data:", data);
} catch (error) {
    console.error("Error fetching protected data:", error.message);
}
};

```

全局处理 Token 过期问题

封装 fetch 工具

比如说前端的 `src/api/customFetch.js`

```

const customFetch = async (url, options = {}) => {
    try {
        const response = await fetch(url, {
            ...options,
            credentials: "include", // 自动携带 HttpOnly Cookie (重要)
        });

        if (response.status === 401) {
            // Token 过期处理
            alert("Session expired. Redirecting to login...");
            window.location.href = "/login"; // 跳转到登录页面
            return;
        }

        return response;
    } catch (error) {
        console.error("Network error:", error.message);
        throw error;
    }
};

export default customFetch;

```

使用封装的工具

在需要的路由页面，导入一下上面封装的工具

```
...
import { customFetch } from "../api/customFetch";

const fetchProtectedData = async () => {
  const response = await customFetch("http://localhost:3000/api/protected",
  {
    method: "GET",
  });

  if (response.ok) {
    const data = await response.json();
    console.log("Protected data:", data);
  }
};
...
```

后端获取 Token

```
// 记得安装上面的 Cookie-parser
// 因为 cookie 存在于 req.headers.cookie 中，而不是直接挂载在 req.cookie 上
const token = req.cookies.authToken; // 从 HttpOnly Cookie 中获取 Token
```

改写代码 - 后端

我们的中间件依赖 Authorization 头中的 Bearer Token。如果你使用的是 HttpOnly Cookie 传递 Token，那么中间件将无法从 Authorization 头中获取 Token。会直接 undefined，之后报错401。

所以，如果前后端通过 HttpOnly Cookie 传递 Token，可以在 req.cookies 中查找 Token

App.js 文件一览

```
const express = require('express');
const bodyParser = require('body-parser');
const cookieParser = require('cookie-parser'); // Cookie 解析
const indexData = require('./routes/siteintro');
const indexRoutes = require('./routes/index');
const articleRoutes = require('./routes/article');
const registerRoutes = require('./routes/register');
```

```

const authRoutes = require('./routes/login');
const adminRoutes = require('./routes/admin');

const cors = require("cors");

const app = express();

app.use(bodyParser.json());
app.use(cookieParser()); // 使用 cookie-parser 中间件

// CORS 配置
app.use(
  cors({
    origin: "http://localhost:5173", // 前端地址
    credentials: true, // 允许携带 Cookie
  })
);

// 路由
app.use('/', indexRoutes);
app.use('/data', indexData);
app.use('/article', articleRoutes);
app.use('/register', registerRoutes);
app.use('/login', authRoutes);
app.use('/admin', adminRoutes);

const PORT = 3000;

app.listen(PORT, async () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

```

修改一下中间件的 Token 获取方式

middlewares/auth.js

```

const { verifyToken } = require('../utils/jwt');

// 中间件: 验证 JWT
const authenticateJWT = (req, res, next) => {
  const authHeader = req.headers.authorization;
  const token = authHeader && authHeader.startsWith('Bearer ')
    ? authHeader.split(' ')[1] // 从 Authorization 头中获取 Token
    : req.cookies?.authToken; // 或从 Cookie 中获取 Token
  // 因为我希望上面两种情况都支持, 所以直接用三元表达式, 如果获取不到 Authorization 的
  // Token, 再从 Cookie 中获取

```

```

    if (!token) {
      return res.status(401).json({ error: 'Unauthorized: No token provided'
});
    }

    try {
      const decoded = verifyToken(token); // 验证 JWT
      req.user = decoded;                // 将用户信息附加到请求对象
      next();                            // 放行
    } catch (err) {
      return res.status(401).json({ error: 'Unauthorized: Invalid token' });
    }
  };

  module.exports = { authenticateJWT };

```

middlewares/verify.js

```

const { verifyToken } = require('../utils/jwt');

function verifyLogin(req, res, next) {
  const authHeader = req.headers.authorization;
  const token = authHeader && authHeader.startsWith('Bearer ')
    ? authHeader.split(' ')[1] // 从 Authorization 头中获取 Token
    : req.cookies?.authToken; // 或从 Cookie 中获取 Token

  // 如果没有 Authorization 请求头, 则视为未登录
  if (!token) {
    console.log('No Authorization header found');
    req.user = null;
    return next();
  };

  // 提取 token (处理 Bearer 前缀)
  // const token = authHeader.startsWith('Bearer ') ? authHeader.split(' ')
  [1] : authHeader;

  try {
    // 验证并解码 token
    const decoded = verifyToken(token);

    // 检查解码后的信息是否完整
    if (!decoded || !decoded.username) {
      console.error('Decoded token missing required fields:', decoded);
    }
  }

```

```
    req.user = null;
    return next();
  }

  req.user = decoded; // 将解码后的用户信息传递到后续中间件
  next();

} catch (err) {
  console.error('Token verification failed:', err.message); // 打印具体错误信息
  req.user = null;
  next();
};
};

module.exports = { verifyLogin };
```

改写代码 - 前端

App.jsx 文件一览

```
import React from 'react';
import { BrowserRouter as Router, Routes, Route } from "react-router-dom";

import Home from './pages/Home';
import About from './pages/About';
import NotFound from './pages/NotFound';
import LoginPage from './pages/Login';
import Admin from './pages/Admin';

import './App.css';

const App = () => {
  return (
    <>
      <Router>
        <div>
          <Routes>
            <Route path="/" element={<Home />} />
            <Route path="/About" element={<About />} />
            <Route path="/Login" element={<LoginPage />} />
            <Route path="/Admin" element={<Admin />} />
            <Route path="/*" element={<NotFound />} />
          </Routes>
        </div>
      </Router>
    </>
  );
};
```

```

        </Routes>
      </div>
    </Router>
  </>
);
};

export default App;

```

/src/api/customFetch.js

```

const customFetch = async (url, options = {}) => {

  try {
    const response = await fetch(url, {
      ...options,
      credentials: "include", // 自动携带 HttpOnly Cookie (重要)
    });

    if (!response.ok) {
      if (response.status === 401) {
        alert("Session expired. Redirecting to login...");
        window.location.href = "/login";
        return;
      } else if (response.status === 403) {
        throw new Error("Access denied: You do not have permission to access this resource.");
      } else if (response.status === 404) {
        throw new Error("Resource not found.");
      } else {
        throw new Error(`Unexpected error: ${response.statusText}`);
      }
    }
  }

  // 自动解析 JSON 数据
  return await response.json();
} catch (error) {
  console.error("Network error:", error.message);
  throw error;
}
};

export default customFetch;

```

/src/pages/Admin.jsx


```

import React, { useEffect, useState } from "react";
import customFetch from "../api/customFetch";

const Admin = () => {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {

    const getUserProfile = async () => {
      try {
        // 用我们封装的 Fetch 去访问需要认证的 私人路由（它的逻辑明确了带上Cookie，如果失效或者未找到，即重定向到 /Login 路由）
        const data = await customFetch("http://localhost:3000/admin", {
          method: "GET" });

        if (!data || !data.user || data.user.role !== true) {
          throw new Error("Access denied: You do not have permission to access this page.");
        }
        setUser(data.user);

      } catch (error) {
        setError(error.message);
        if (error.message.includes("Session expired")) {
          window.location.href = "/login";
        }
      } finally {
        setLoading(false);
      }
    };

    getUserProfile();

  }, []);

  if (loading) return <div>Loading ...</div>;
  if (error) return <div className="text-red-500">Error: {error}</div>;

  return (
    <div>
      <h1>Admin Dashboard</h1>
      <p>Welcome, {user?.username}!</p>
      <p>Your role: {user?.role ? "Admin" : "User"}</p>
    </div>
  );
};

```

```
    </div>
  );
};

export default Admin;
```

书接上文

像后端一样接收参数(params)! (v1.5) 2025.08.29

前端路由完全可以像是后端一样接收params，也就是 `/:key` 这种路由，后端通过 `const key = req.params.key;` 的方式获得变量。其实在前端也支持这样的操作，在接收到这样的 `params` 变量后，就可以设计代码逻辑去 Fetch 不同的后端路由，或者是渲染不同的内容。

首先我们在 `App.jsx` 里面定义我们的路由，我们要这样定义

```
import Product from './pages/Products';
import Category from './pages/Category';
...
...
<Route path='/product/:key' element={}<Product /> } />
<Route path='/category/:key' element={}<Category /> } />
...
```

而这个 `Key(/:key)` 就会传给 `element` 的模版中，这个 `Key` 并不是固定写法，如果你希望叫 `id`, `name` 之类的也是完全可以的，随心就行。

此时我们只需要访问 -> `http://网页/product/123`

就会往这个页面(路由)内传入一个`key`，也就是123，之后我们在 如`Product`模版 中如何接收这个变量呢？

`Product.jsx`

```
...
import { useParams } from 'react-router-dom'; // 导入的 useParams 就是接收
Params所必须的，它由react-router-dom提供

...
function Product() {
  // useParams 钩子用于从 URL 中获取动态参数。
```

```
// 在这里，它获取 URL 路径中的 'key' 参数，例如 /product/123 中的 '123'。  
const { key } = useParams();  
...  
...  
}  
  
export default Product;
```

就这样，此时就接收到 `key = 123` 了，就那么简单，`const { key } = useParams();`

后记

我相信代码已经没什么好说的了，并且我在网站已经提供了整个项目的文件打包，包括代码。

由 Dontalk 会员 - 【葵】 耗时4天编写 - 如有不对，敬请原谅和指正。

如果你希望下载本文 PDF 以及配套代码，请前往「dl.dontalk.org」找到「现代网页开发入门指南书 - 从数据库设计，到后端，再到前端」即可下载。或者博文提到的 GitHub地址 也可以下载到副本 (包括本指南书的PDF档)。

Dontalk 地址: dontalk.org

Github 仓库地址(25年5月中): <https://github.com/infoabcd/Modern-Web-Development-Beginner-s-Guide/>

但是请注意，本作品采用 [署名-非商业性使用 4.0 国际 (CC BY-NC 4.0)] 协议发布。

你可以自由转载、分发、分享，但必须注明作者及出处，不得用于任何商业用途。

否则在沟通无果后，Dontalk 团队不介意采用法律武器捍卫权益。

协议详情请见: <https://creativecommons.org/licenses/by-nc/4.0/deed.zh>